Information and Software Technology 194 (2026) 108071

Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof

Evaluation of the Impact of Image Mutations on the Origin Classification of Digital Images

Vaishnav Koka[1], Ramanand[2], Shouvick Mondal[*,1], Yogesh Kumar Meena[2]

Indian Institute of Technology Gandhinagar, Gujarat, India

ARTICLE INFO

Dataset link: https://osf.io/fau7k

Keywords:
Classifier
Discrete mutations
Continuous mutations
Black box testing
Permutations
Accessibility

ABSTRACT

Context: Image origin classifier tools are increasingly deployed to verify the origin and authenticity of digital images, especially for detecting AI-generated or manipulated content. Despite their usage in forensic and media applications, the robustness of these systems under real-world image perturbations remains largely unexplored.
Objective: This study aims to evaluate the robustness of a proprietary tool (developer claimed consistency >90%) against a wide spectrum of systematic image mutations to uncover biases, failure points, and inconsistencies in performance.
Methods: We designed a large-scale mutation-driven automated testing approach using `ImageMagick`, an open-source software suite for image processing, to apply 128 distinct mutation types — categorized into 101 discrete and 27 continuous filters — across a dataset of 40 original images, yielding over 31,000 mutated samples. The classifier's output was collected via an automated Python pipeline and analyzed after parsing structured data from the tool's JSON responses.
Results: The classifier showed significant sensitivity to specific mutation types, decreasing consistency to 61% under discrete mutations and dropping further to 30%–41% under continuous parameterized mutations. Mutations involving grayscale conversions and rotations were particularly detrimental. Additionally, performance degraded with lower resolutions, with a critical threshold observed to be ~280 pixels in height or width.
Conclusion: Our empirical findings underscore black-box-based image classification systems' vulnerability to structured image-level mutations, without requiring access to their internal logic. Building on this observation, the proposed mutation-based methodology provides a generic, automated, scalable and reusable framework for systematically generating diverse visual transformations for different image-based testing strategies.

1. Introduction

Digital image mutation refers to the process of applying systematic transformations in a controlled manner to create variations of the original image. These mutations can include basic pixel-level changes (e.g., brightness adjustment, contrast modification) and geometric alterations (e.g., flipping, rotating, cropping). Data augmentation, a crucial machine learning technique that involves artificially expanding datasets to expose models to a variety of perturbations on the training data, makes extensive use of image mutation. Image mutations are usually used to simulate realistic perturbations in order to evaluate the robustness of systems that process visual inputs. In the context of Visual Deep Learning (VDL) systems, such mutations help evaluate how models respond to variations in input data—identifying failure cases through input variation, without accessing or altering the model's internal structure.

1.1. Definitions and preliminaries

Following are some of the terms and definitions that are used across this article:

1.1.1. Digital image

A digital image is a collection of pixels (picture elements). Each pixel represents a single point of visual information. Depending on the type of image, these pixels hold specific data, such as color information in the case of color images or intensity values for grayscale images. They typically have three channels such as Red (R), Blue (B), Green

* Corresponding author.
E-mail addresses: vaishnav.koka@iitgn.ac.in (V. Koka), ramanand.k@iitgn.ac.in (Ramanand), shouvick.mondal@iitgn.ac.in (S. Mondal), yk.meena@iitgn.ac.in (Y.K. Meena).
[1] Associated with the Software Engineering and Testing (SET) Group, IIT Gandhinagar, India
[2] Associated with the Human-AI Interaction (HAIx) Lab, IIT Gandhinagar, India

https://doi.org/10.1016/j.infsof.2026.108071
Received 15 September 2025; Received in revised form 13 January 2026; Accepted 9 February 2026
Available online 23 February 2026

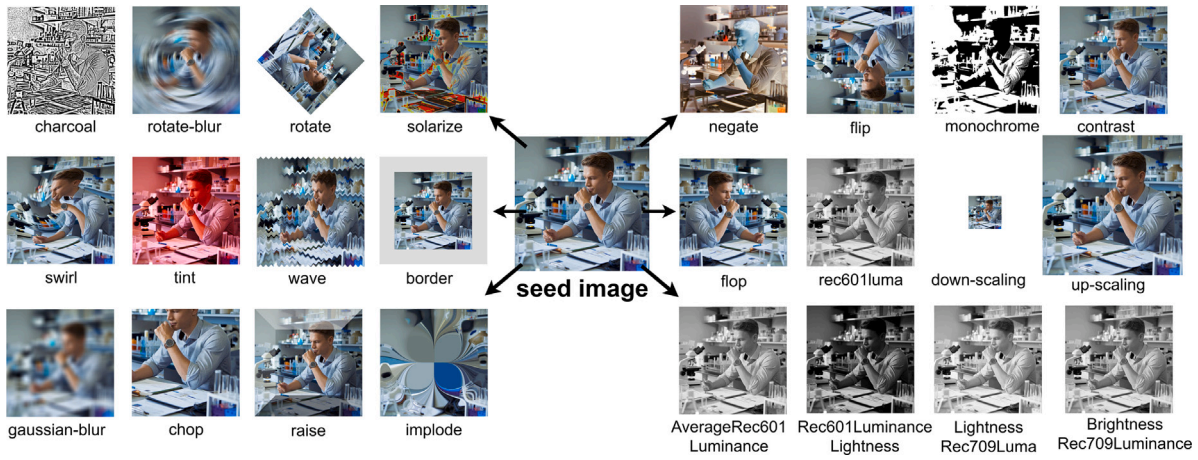


Fig. 1. Mutation space visualization centered around a seed image. The central image represents the original unmodified input. Surrounding it are systematically generated variants using a range of transformations. Mutations are grouped as follows: *Continuous mutations* – `charcoal`, `rotation-blur`, `rotate`, `solarize`, `threshold`, `border`, `chop`, `edge`, `emboss`, `gaussian-blur`, `implode`, `paint`, `posterize`, `spread`, `swirl`, `tint`, `wave`; *Discrete mutations* – `flip`, `flop`, `negate`, `rgb`, `contrast`, `monochrome`, `normalize`, `rec601luma`; *Combinatorial mutations* – `Average_Rec601Luminance`, `Brightness_Rec709Luminance`, `Lightness_Rec709Luma`, `Rec601Luminance_Lightness`, `Rec601Luminance_MS`; and *Scaling transformations* – `up-scaling`, `down-scaling`. Each image is annotated with the applied transformation, highlighting the diversity of perturbations in our mutation-driven testing framework [1].

(G) and dimensions such as Height (H), Width (W) and Channels (C) [2]. These tiny elements come together to form the complete visual representation that we perceive as an image. The total number of pixels in a digital image, measured in terms of both width and height, is known as its resolution. An image with a resolution of 1920 × 1080, for instance, has 1920 pixels in width and 1080 pixels in height, for a total of more than two million pixels. The image's clarity and amount of detail are determined by its resolution; finer detail is usually seen at higher resolutions. Together, the value and arrangement of these pixels determine how the digital image looks.

1.1.2. Image mutations

In our study, an image mutation (perturbation) refers to a systematic transformation applied to an original (seed) image to generate a variant. Mutations are categorized into two primary types based on their behavior: discrete and continuous. A possible mutation space for a seed image is shown in Fig. 1.

1.1.3. Discrete mutation

In this work, Discrete mutations involve command-based transformations that result in abrupt and fixed changes to the image. These include operations like flipping, cropping, grayscale conversion, etc. Each discrete mutation operates independently and applies an immediate, non-parametric change to the image.

1.1.4. Continuous mutation

Continuous mutations, as applied in our study, refers to parameterized operations that allow for smooth and gradual transformations of visual features. These include variations in brightness, contrast, sharpness, noise levels, and more. These mutations are controlled by numeric parameters that define the degree of change.

1.1.5. Image scaling

In the scope of this study, scaling refers to systematic changes in image resolution, where the spatial dimensions of an image (height and width) are scaled either down (down-scaling) or up (up-scaling).

1.2. Challenges in image origin classification

Image origin classifiers play a critical role in ensuring the authenticity of images [3]. However, their effectiveness depends on their ability to correctly classify images under various conditions, including those with mutations. For instance, a classifier that is not trained on a diverse set of image mutations may produce misclassifications, which can have serious real-world implications. For example, research has shown that small perturbations — such as stickers or graffiti — can cause traffic sign classifiers in autonomous vehicles to misidentify stop signs as speed limit signs, potentially leading to traffic violations or accidents [4].

In medical imaging, adversarial perturbations applied to medical images — particularly those assessing tumor burden in radiological scans — have been shown to cause high-performing diagnostic systems to misclassify malignant findings as benign, raising critical concerns about treatment delays and manipulation of clinical trial outcomes [5]. Similarly, in facial recognition security systems, an AI model that misidentifies individuals due to lighting variations or slight pixel alterations could lead to false arrests or access denials [6]. Another example in facial recognition, academic studies have revealed severe demographic bias: commercial systems exhibited error rates up to 30% in classifying darker-skinned females—leading to wrongful identifications [7]. In e-commerce, research shows that despite deep learning optimizations, product image classifiers struggle with similar-looking products and require complex model fusion to reduce misclassification rates [8].

These examples highlight how even subtle image perturbations can lead to critical failures across domains. To minimize misclassification risks in image origin classifiers specifically, it is essential to train and evaluate these systems on datasets that incorporate a diverse range of image mutations. By systematically applying the image mutations described earlier, our framework allows for evaluating the robustness of image origin classifiers against visual perturbations. While this does not directly improve model resilience, it helps identify specific failure cases. A reliable classifier should ideally maintain consistency in the face of such mutations, especially in critical real-world applications where decisions are made based on image authenticity. A well-trained classifier should be capable of distinguishing genuine content from altered or synthetic images with high consistency, ensuring reliability across critical applications.

Presently, the curation of datasets to test the classifier plays an important role; there are no clearly specified steps to mutate or transform the images to test the classifier model; the applicability and quality of Multi-modal large language model (MLLM)-based mutations are unexplored, and they often struggle to edit existing image semantics effectively. The most common data augmentation operations are crop, flip, and rotate [9]. Circular shift is another method of mutating images and works well when combined with the flip operation. Most existing methods primarily mutate some pixel-level semantics, such as brightness and contrast [10]. The approaches to mutating images are limited and not controlled. In our study, we have identified 128 different ways to mutate a seed image, where one seed image can produce 792 image mutations ranging from basic operations to high-level mutations.

1.3. Can digital image mutations cause misclassifications?

To understand how existing image origin systems behave under such extensive perturbations, we referred to as closed-source proprietary image origin classifier [11] (denoted as Tool T). To the best of our knowledge, this tool was developed to identify whether a given image was generated by a specific diffusion-based model. According to the developers' internal evaluations, Tool T reported over 90% consistency in identifying unmodified generated images and showed similar performance even after standard image alterations such as cropping, resizing, and JPEG compression

However, these tests were limited to conventional transformations, and the tool's behavior under more diverse or semantically meaningful mutations—such as those in our curated mutation space—remains unknown. Importantly, its predictions reflect probabilistic judgments rather than definitive evidence of generative origin. Our study is motivated by this gap and aims to examine the classifier's robustness using a systematically mutated dataset far beyond standard augmentation techniques. In this work, we performed a study on robustness testing of the Image origin classifier using systematic input image mutations. The classifier showed significant sensitivity to specific mutation types, decreasing consistency to 61% under discrete transformations and dropping further to 30%–41% under continuous parameterized mutations. Mutations involving grayscale conversions and rotations were particularly detrimental. Additionally, performance degraded with lower resolutions, with a critical threshold observed to be around ∼280 pixels in height or width. Our evaluation is driven by the following research questions (RQs).

RQ1 (mutation operators): *What are the different ways through which input images can be mutated for feeding the Image origin Classifier?* We formulate how to apply the mutation (perturbation) on the seed (clean) image under classification. These operators can be discovered by systematically mutating the input such that the output of the classifier changes. For example, decreased brightness/scale of the image, binarisation, in-painting, noise injection, etc.

RQ2 (mutation spectrum): *Do different mutation operators affect the classifier's output differently?* We identify the strengths of different mutation operators based on the percentage change in the classifier's continuous numerical score regardless of the binary classification. Interesting cases will be those edge (corner) cases that toggle the binary classification with a small change in the input. This would mean that a mild mutation may adversely affect the classifier's performance.

RQ3 (mutation combinatorics): *Does the multiplicity and order of mutations play a role in the classifier's performance?* After identifying individual mutation operators and observing their isolated effects, we investigated whether the order in which different operators were applied — especially those from distinct mutation categories — affected the classifier's output. Our results confirm that both the sequence and combination of mutations can lead to significant variations in output, underscoring the importance of combinatorial factors in evaluating such systems.

Contributions: our main contributions in this paper are as follows:

- We developed a mutation-driven automated testing framework to generate a large-scale perturbed dataset for robustness assessment and evaluated a closed-source proprietary Tool (T) (developers' claimed consistency > 90%), under systematic dataset variations.
- We identified 128 image mutation operators classified into two categories based on their parameter tunability: (i) discrete (fixed mutation) - 101, and (ii) continuous (tunable mutation) - 27, enabling systematic dataset mutation and expansion. These operators generated 792 variants per seed image, expanding the initial 40 images into over 31,000 mutated samples ($\approx 4,000$ discrete and $\approx 27,000$ continuous).
- Results show that the Tool (T)'s consistency dropped to 61% under discrete mutation and 30%–41% under continuous mutation operations, performance degraded with lower resolutions, with a critical threshold observed to be ∼280 pixels in height or width.
- Our interpretive analysis reveals that applying SSIM-based similarity [12,13] thresholding together with Wilson score-driven [14] error ranking provides insight into the relative effectiveness of mutation operators, which can be leveraged for dataset augmentation, particularly when the data was imbalanced and the number of samples was relatively small.
- To foster reproducibility and further research, we provide the complete dataset used in our experiments, including original input images, their corresponding mutated outputs, and a structured spreadsheet file documenting the applied transformations. The artifacts are publicly available at: https://osf.io/fau7k.

The rest of this paper is organized as follows: Section 2 elaborates the methodology of our study. Section 3 outlines the experimental protocol. Section 4 presents results. In Section 5 we discuss downstream users of our work. Section 6 reviews related work. Section 7 explains threats to validity. Section 8 discusses the future scope and finally, Section 9 concludes the paper.

2. Testing methodology

In this section, we describe the workflow of our dataset preparation and mutation operators applied to them, resulting in transformed and mutated images that are sent to the tool's API using the `curl` command-line interface (CLI) for evaluation; the results gets stored in CSV files which in turn are analyzed for patterns, correlations, safety issues and finally highlight the classifier performance.

2.1. Testing pipeline

The pipeline involves preparing the dataset, establishing a reliable ground truth, applying targeted mutations using the Tool (T), integrating it into a structured testing framework, and systematically collecting outputs for analysis. Our workflow is illustrated in Fig. 2.

2.1.1. Dataset preparation

The dataset used in this research comprises 40 images sourced from various online repositories, representing a diverse range of categories such as AI-generated, vintage, anime, child abuse, child labor, abstract design, drugs, food, war, medical imagery, and depictions of violence, but not limited to war. The dataset comprises both real-world and synthetically generated images, enabling a comprehensive evaluation of the classifier's performance under diverse conditions. These images were employed to assess the model's ability to accurately determine the `IsGeneratedByTool` attribute.

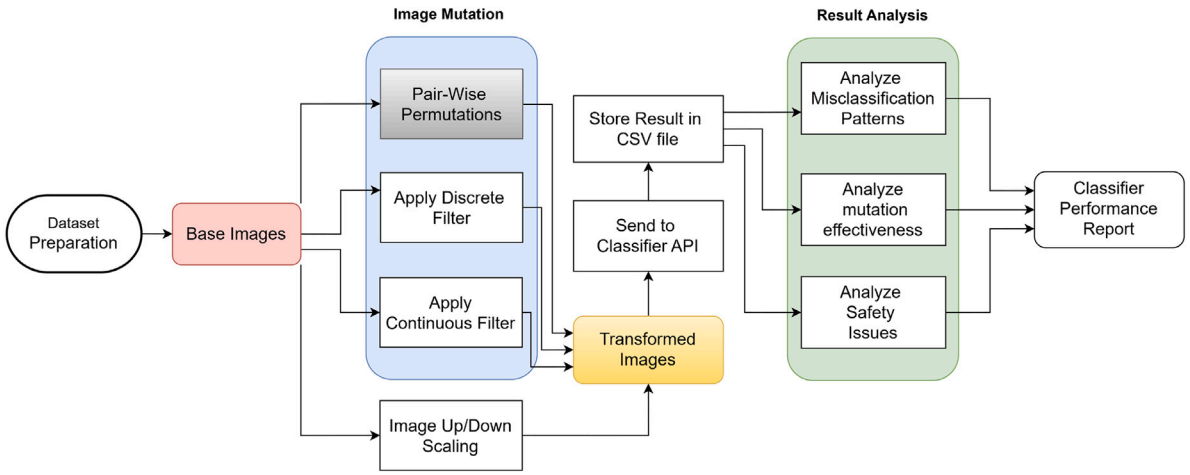


Fig. 2. Our Robustness Testing Workflow of the Image origin classifier.

2.1.2. Constructing the ground truth

To evaluate the robustness of the image-origin classifier under mutation-based testing, no external or re-labeled ground truth is introduced. Instead, the classifier's prediction on each original (seed) image is treated as a baseline reference. Seed images are first processed by the classifier API, and the resulting outputs are recorded as baseline predictions. For each mutated image derived from a given seed, the classifier's output is compared against the corresponding baseline prediction. A prediction is considered correct if the output on the mutated image matches the model's prediction on the original seed image; any deviation is treated as a misclassification [15]. This formulation measures prediction consistency under controlled, semantic-preserving perturbations rather than absolute classification consistency with respect to labels. Following established practices in metamorphic and mutation-based testing [16,17], this approach avoids reliance on external ground truth and does not introduce circular validation. Instead, robustness is evaluated by measuring the stability of the classifier's decision process under controlled, semantic-preserving image mutations relative to its own baseline predictions, rather than real-world classification correctness.

2.1.3. Tool used-image mutations

General-purpose mutation tools, such as `Radamsa` [18], operate as "black-box" systems that lack format-specific knowledge, applying random modifications like bit flips to any input, whether text or binary. This approach alters the structure of the input in unpredictable ways. As a result, when used with images, such tools often produce outputs that are unreadable or corrupted because they are not aware of image-specific constraints or semantics. Without the ability to perform targeted mutations — such as controlled color adjustments or format-preserving transformations — these tools are unsuitable for scenarios that require well-formed, visually coherent image mutations. For our work, where meaningful and reproducible image-specific changes are essential, a domain-aware tool is necessary. Therefore, to rigorously test the classifier, we employed `ImageMagick 7.1.1-41`, a powerful open-source software suite for image processing, which is particularly beneficial for the project as it aligns with the need for reproducibility and adaptability in testing environments geared toward research. Our objective was to perform controlled, interpretable, and image-specific mutations such as contrast shifts, color space changes, blurs, flips, and resolution scaling etc. This level of control is essential for pruning the mutation space meaningfully and maintaining semantic validity in visual input. `ImageMagick`, with its deterministic filters and reproducible CLI utilities, allowed us to automate the creation of thousands of mutated images while preserving image structure, enabling consistent, reproducible evaluation of the classifier's robustness.

In the context of robustness testing for the classifier, the primary objective was to assess the classifier's ability to handle variations and perturbations (deviations) on seed images that simulate real-world changes, such as lighting variations, scale changes, rotation, etc. Adversarial conditions, on the other hand, involve intentional modifications to images designed to deceive or mislead machine learning models. These modifications are often subtle and not easily noticeable to humans, but can cause significant errors in AI systems.

To construct a diverse and systematic mutation framework, we leveraged the transformation capabilities of `ImageMagick` due to its extensive support for parameterized operations, known as *image operators*. These operators include transformations such as rotation, scaling, noise injection, and brightness adjustment, all of which can be applied independently and do not persist across command-line sessions. This characteristic was essential for our use case, where each image mutation needed to be isolated to prevent cumulative side effects across tests. The selection of mutation operators used in this work was guided by two primary criteria. First, the operator must act directly and exclusively on the image data, rather than persist as a session-wide configuration. Second, the operator must allow isolated, one-time changes, ensuring that each mutation is applied independently. To this end, we excluded *image settings* — persistent configurations that influence multiple operations — as their use would introduce unintended dependencies across tests and compromise reproducibility, as once an image setting is applied, it stays in effect until it is explicitly reset or the command terminal session ends. Accordingly, we curated a total of 128 mutation operators grouped into 101 discrete and 27 continuous mutation operators. These represent a wide operational space but are not exhaustive of all possible manipulations. Rather, they reflect a targeted subset of meaningful, image-specific mutations suitable for testing Tool (T) under controlled conditions. This design choice defines the scope of our mutation study and also introduces a natural limitation: transformations outside `ImageMagick's` operator set were not considered. Thus, our methodology prioritizes isolated, reproducible, and semantically diverse mutations, implemented through non-persistent image operators, aligning well with the goals of evaluating robustness in a black-box classification setting.

Based on the inclusion criteria discussed above, we systematically categorized the mutations in a tabular format. To better illustrate the structure and content of the mutation table, we provide a brief explanation of selected rows from Table A.6. This table presents the complete list of image mutation operations applied in this study. Each row in the table corresponds to a specific image mutation applied using `ImageMagick` commands. The table is divided into two main sections: **discrete mutation operators** (S.No 1–9), which represent fixed,

command-based transformations that produce immediate changes without parameterization. For example, in the discrete section, row 1 demonstrates the use of the `-colorspace` CMY command, which converts the image into the CMY (Cyan, Magenta, Yellow) color space. This color model is subtractive in nature and primarily used in printing contexts. In addition, geometric transformations such as `-flip` (vertical inversion) and `-flop` (horizontal inversion) are included as individual mutation operators. These operations preserve the image content while altering its orientation. The `-monochrome` filter, on the other hand, binarises the image into black and white by thresholding pixel intensity, discarding all color and grayscale information. In contrast, **continuous mutation operators** (S.No 10–36) apply pixel-based operations that gradually modify an image's attributes, such as sharpness, contrast, texture, and structural elements, enabling a more detailed mutation of images. These filters are particularly useful for fine-tuning effects and testing how changes in parameters influence the resulting image. For example, in the continuous section, row 18 demonstrates the use of the `-rotate` command, which applies rotation to the image by a specified degree. This operation changes the image's orientation while preserving its overall content. In addition, the `-blur` operation introduces a smoothing effect by reducing high-frequency pixel variations, which mimics out-of-focus or motion-blurred conditions. The `-tint` filter overlays a specified color onto the image while maintaining luminance, altering the perceived color balance without distorting spatial structure. The `-chop` command removes rectangular regions from the image, effectively modifying its composition. These transformations were applied systematically to the dataset to evaluate the classifier's behavior in response to diverse image-level perturbations.

2.1.4. Creation of testing framework

The mutation framework in this study was developed to simulate a broad spectrum of image alterations for evaluating the classifier. Mutation operators were systematically categorized into two primary classes: (i) discrete operators, which apply fixed transformations such as `grayscale`, `negate`, `monochrome`, `ICC profiles`, `flip`, `flop` etc., and (ii) continuous operators, which involve tunable parameters, such as varying degrees of `rotate`, `gamma`, `implode`, `white-threshold`, `blur`, `chop`, `solarize` etc. In addition to these mutation strategies, both image upscaling and downscaling operations were incorporated to assess the classifier's sensitivity to resolution changes, which are commonly observed in real-world image processing pipelines.

All mutation and scaling operations were applied to a consistent dataset of original images, ensuring uniformity across experimental conditions. To manage the diversity of transformations, each mutation was encoded as a key–value pair within a dictionary structure, where the key represented the type of mutation and the value captured its configuration parameters. To maintain clarity and support reproducibility, the output images corresponding to each mutation were saved in separate, well-structured directories to prevent any ambiguity during evaluation. This organizational strategy was critical to maintain traceability and avoid ambiguity during the analysis of results.

The mutated images were passed as input to the classifier via the CLI utility `curl`, which served as an interface to interact with API calls from the classifier. The classifier's output, returned in JSON format, was programmatically captured and converted into structured CSV files for further analysis. Given that both image processing (via `ImageMagick`) and API invocation are CLI-based, the entire workflow — including image mutation, API interaction, and result logging — was fully automated using custom Python scripts. The scripts allow us to run the same set of mutations on different datasets or repeat the tests under the same conditions, ensuring that the results are comparable and reproducible. This end-to-end automation enabled exhaustive and reproducible testing at a large scale, while also reducing the risk of manual errors in the evaluation process and ensuring consistency throughout the experimental pipeline.

2.1.5. Result collection from API

The `curl` CLI tool was utilized to systematically process the mutated images through the classifier API. This approach allowed for efficient, repeatable interaction with the classifier, enabling high-throughput testing across a wide range of image mutations. For each image processed, the classifier returned a structured JSON response as output. This enables quantitative analysis of classifier behavior across different mutation types and supports deeper insights into model reliability and performance. These provided valuable numerical insights into the internal behavior of the classifier, which otherwise operates as a black-box system. By aggregating and analyzing the classifier's responses across different mutation types, it was possible to observe how specific filters — categorized as discrete and continuous — affected the model's consistency and classification. This correlation between classifier outputs and image-level perturbations enabled a more nuanced understanding of its sensitivity. Such analysis is essential for uncovering edge cases where the classifier may fail silently or exhibit low confidence or consistency—scenarios that carry significant implications for the reliability of image origin systems, particularly in high-stakes applications.

3. Experimental setup

In this section, we have specified the hardware and software requirements to test the classifier.

3.1. Runtime environment

The experiments were conducted on a system running `Ubuntu 22.04 LTS`, equipped with the following hardware and software configurations: (i) Processor: Intel Xeon Gold 5318Y (24 cores/48 threads) @ 2.10 GHz, (ii) RAM: 256 GB DDR4-3200, (iii) Chipset: Intel C621, (iv) Image processing software: `ImageMagick v7.1.1-41`, and (v) Programming environment: `Python 3.14`. All mutation operations and batch processing scripts were executed using this configuration to ensure consistency and reproducibility across the evaluation pipeline.

3.2. Dataset mutation

Table A.6 presents the complete list of image mutation operations applied in this study. The mutations are categorized into two main types: **discrete mutation operators** (S.No 1–9) and **continuous mutation operators** (S.No 10–36).

3.2.1. Discrete filters

Terminology Clarification. In this study, the terms *mutation operator* and *filter* are used interchangeably. Both refer to specific image-level transformations applied during the mutation testing process. In particular, discrete filters refer to fixed-command transformations that induce immediate, non-persistent modifications to images using `ImageMagick`'s CLI. These operations are stateless—they affect only the image currently being processed on the CLI and do not carry over across multiple operations. Such filters are particularly effective for generating clear and distinct variations in image attributes, making them suitable for controlled mutation experiments. A total of 101 discrete filters were applied such as `colorspace`, `flip`, `flop`, `ICC profile`, `negate`, `grayscale`, `monochrome`, and `contrast`, among others. When these filters were applied to 40 images from the original dataset, the process resulted in a total of 4040 uniquely mutated images, forming a substantial subset for evaluating the Tool (T)'s response to discrete, well-defined image mutations.

3.2.2. Continuous filters

Continuous filters apply pixel-based operations that gradually modify an image's attributes, such as sharpness, contrast, and rotation, enabling a more detailed mutation of images. They operate using numerical parameters, which can range from single values to multiple dimensions. These filters are particularly useful for fine-tuning effects and testing how changes in parameters influence the resulting image. They are characterized by their reliance on numeric input parameters, provided a dynamic approach to mutating images. These filters not only allowed fine-tuned variations but also facilitated testing beyond standard boundaries, offering a robust means to evaluate `ImageMagick's` adaptability when performing image transformations. In addition to discrete filters, 27 continuous filters were applied to the same dataset of 40 seed images. The number of ways each filter accepted inputs varied between one and six, offering diverse transformation capabilities. The images were mutated and tested beyond their defined boundary values, simulating extreme scenarios.

3.3. Input to the Tool (T)

The Tool (T) receives mutated images as input to assess their origin and authenticity. These images undergo specific transformations before being analyzed. The Tool (T) then processes them to generate origin results.

3.4. Output from the Tool (T)

The Tool (T) generates results in JSON format, which are later converted into CSV for analysis. Each image in the output contains key attributes that help in understanding its numerical value and potential risks.

Key fields in the output

- **Name** – The name of the mutated image file, formatted to reflect both the original image and the specific mutation applied. This naming convention enables clear traceability between the seed image and the transformation used to generate the mutated version.
- **Unique ID** – A unique identifier assigned to each mutated image, ensuring consistent reference and traceability throughout the evaluation process.
- **Transformation** – A numerical score provided by the Tool (T), representing an internal measure related to the image's origin or the extent of transformation applied.
- **Confidence** – An additional numerical value that may reflect the Tool (T)'s confidence or other origin-related properties associated with the image data.
- **IsGeneratedByTool** – A boolean flag indicating if the image was generated or modified by a Tool (T).
- **Is_unsafe** – A boolean flag indicating whether the Tool (T) has identified the image as potentially unsafe based on classification criteria.

3.5. Evaluation metrics

The proposed metric for quantifying classifier performance is as follows:

- **Detection Consistency (%:):** This metric quantifies the stability of the Tool (T) for the given mutation, calculated as:

$$\text{Consistency}(m) = \frac{1}{|\mathcal{I}|} \sum_{\text{img} \in \mathcal{I}} \mathbb{I}\big(\text{R}(\text{img}, m) = \text{GT}(\text{img})\big) \times 100 \tag{1}$$

Example:. Suppose we apply the mutation `-rotate 90` to a set of 10 images. Tool T makes predictions on each of the rotated images. Out of the 10 predictions, 8 match the ground truth labels. Therefore, the detection consistency for this mutation is calculated as:

$$\text{Consistency}(-\texttt{rotate } 90) = \frac{8}{10} \times 100 = 80\%$$

So, the Detection Consistency for the `-rotate 90` mutation is 80%.

- **Aggregate Consistency (%:)** This denotes overall detection consistency computed across all mutations belonging to a mutation class C.

$$\text{Consistency}(C) = \frac{1}{|M^C|} \sum_{m \in M^C} Acc(m) \times 100 \tag{2}$$

Example:. Consider a mutation class *Grayscale* containing the following three operators: `-grayscale Rec601Luma`, `-grayscale Lightness`, and `-grayscale Average`. Suppose their respective detection consistencies are 90%, 85%, and 95%. The aggregate consistency for this mutation class is computed as:

$$\text{Consistency}(\text{Grayscale}) = \frac{1}{3}(90 + 85 + 95) = \frac{270}{3} = 90\%$$

So, the Aggregate Consistency for the Grayscale mutation class is 90%.

Where:

- M^C : The set of all mutations in class C.
- $|M^C|$: The cardinality of the set M^C.
- img : The input image under evaluation.
- $\mathcal{I}$: The set of all images.
- $|\mathcal{I}|$: The total number of images (cardinality of the set Images).
- R(img, m) : The Tool (T)'s prediction for image img under mutation m.
- GT(img) : The ground truth class for image img.
- $\mathbb{I}$: An indicator function that equals 1 if the condition inside is true, and 0 otherwise.

3.5.1. Structural SIMilarity (SSIM) Index

In the scope of this study, the Structural Similarity Index (SSIM) [12] is employed as a perceptual similarity measure to quantify the degree of structural resemblance between an seed image and its mutated counterpart. SSIM evaluates image similarity by jointly considering luminance, contrast, and structural information, thereby enabling controlled filtering of mutations that preserve visual structure while introducing systematic perturbations. In this work, SSIM serves as a criterion for threshold-based selection of mutated images (samples) during interpretive analysis.

SSIM Score (%): [12] This denotes the overall consistency for mutation class P, computed as the average detection consistency over all mutation operators belonging to that class. All images were converted to grayscale and resized to a uniform resolution of 512 × 512 pixels prior to SSIM computation. Grayscale conversion removes chromatic variation and enables structural similarity to be assessed independently of color information, while fixed-size resizing ensures consistent comparison across all image pairs. Fixing the input resolution avoids confounding effects arising from resolution-dependent behavior and enables fair, controlled comparison of prediction consistency across mutations. The selected resolution provides sufficient structural detail while maintaining computational efficiency and is commonly adopted in image-based robustness studies. Although other image resolutions could be explored, this study fixes the image size to isolate the effect of mutation operators under controlled conditions. The structural similarity index (SSIM) was used to evaluate perceptual similarity between the seed and mutated images. SSIM was computed locally using a sliding window, as the metric evaluates structural similarity over small

image regions, which is consistent with its standard formulation and implementation and the final score was obtained by averaging the local SSIM values over the entire image.

$$\mathrm{SSIM}(x, y) = \frac{(2\mu_x\mu_y + P_1)(2\sigma_{xy} + P_2)}{(\mu_x^2 + \mu_y^2 + P_1)(\sigma_x^2 + \sigma_y^2 + P_2)} \tag{3}$$

In this formulation, μ_x and μ_y denote the mean intensities of images x and y, σ_x^2 and σ_y^2 denote their variances, and σ_{xy} represents the covariance between the two images. The SSIM metric measures perceptual similarity by jointly capturing luminance, contrast, and structural correspondence, making it more aligned with human visual perception than pixel-wise error measures.

The constants P_1 and P_2 are small positive stabilization terms that prevent numerical instability when the denominators approach zero. In our experiments, SSIM was computed using a standard reference implementation that internally defines these constants according to the original formulation by Wang et al. [12]. We do not manually specify or tune these constants, as our analysis focuses on relative structural similarity trends across image mutations.

Example:. To illustrate the numerical computation of the Structural Similarity Index (SSIM), we consider a small 2×2 grayscale image patch x extracted from a seed image and the corresponding patch y from its mutated counterpart, where the numerical entries represent pixel intensity values. While SSIM is typically evaluated over larger sliding windows, this simplified example is used solely to aid clarity of exposition. The numerical illustration follows the standard special-case formulation introduced by Wang et al. [12] and employs the same notation as the original definition, serving only as a conceptual demonstration.

$$x = \begin{bmatrix} 120 & 125 \\ 130 & 135 \end{bmatrix}, \quad y = \begin{bmatrix} 122 & 123 \\ 128 & 137 \end{bmatrix}$$

First, compute the mean pixel values:

$$\mu_x = \frac{120 + 125 + 130 + 135}{4} = 127.5, \quad \mu_y = \frac{122 + 123 + 128 + 137}{4} = 127.5$$

Next, compute the variance of each image:

$$\sigma_x^2 = \frac{(120 - 127.5)^2 + (125 - 127.5)^2 + (130 - 127.5)^2 + (135 - 127.5)^2}{4} = 31.25$$

$$\sigma_y^2 = \frac{(122 - 127.5)^2 + (123 - 127.5)^2 + (128 - 127.5)^2 + (137 - 127.5)^2}{4} = 33.25$$

Then, compute the covariance between x and y using the `split` environment for clarity:

$$\begin{aligned}\sigma_{xy} = \frac{1}{4}\Big[&(120 - 127.5)(122 - 127.5) + (125 - 127.5)(123 - 127.5) \\ &+ (130 - 127.5)(128 - 127.5) + (135 - 127.5)(137 - 127.5) \Big] \\ &= 32.25\end{aligned}$$

Assume constants $P_1 = 1$ and $P_2 = 1$ (to stabilize division). The SSIM index is computed as:

$$\mathrm{SSIM}(x, y) = \frac{(2\mu_x\mu_y + P_1)(2\sigma_{xy} + P_2)}{(\mu_x^2 + \mu_y^2 + P_1)(\sigma_x^2 + \sigma_y^2 + P_2)}$$

Substituting the computed values:

$$\mathrm{SSIM}(x, y) = \frac{(2 \cdot 127.5 \cdot 127.5 + 1)(2 \cdot 32.25 + 1)}{(127.5^2 + 127.5^2 + 1)(31.25 + 33.25 + 1)} \approx 1.0$$

This simplified example yields an SSIM close to 1, indicating high structural similarity. For larger images, the computation follows the same principle but is performed over all pixels or sliding windows.

Algorithm 1 Mutation Operator Analysis Using SSIM Thresholding and Wilson Error Estimation

Require: Dataset D consisting of tuples $(x, y, \hat{y}, m, s)$, where x is a mutated image, y the ground-truth label, $\hat{y}$ the model prediction, m the applied mutation operator, and $s = \mathrm{SSIM}(x, \mathrm{seed}(x))$

Require: SSIM threshold set $\mathcal{T} = \{0.50, 0.55, 0.60, \ldots, 1.00\}$ ▷ Explicitly defined as an SSIM threshold

Require: Confidence parameter $z = 1.96$ (95% Wilson interval) ▷ Provides a statistically robust minimum error estimate per operator

Ensure: Consistency history matrix $H(m, \tau)$ for the most harmful mutation operators ▷ τ denotes the minimum SSIM similarity between a mutated image and its corresponding seed image required for inclusion in the evaluation.

Step 1: Mutation Ranking at Base SSIM Threshold

$D_{0.50} \leftarrow \{x \in D \mid \mathrm{SSIM}(x, \mathrm{seed}(x)) \geq 0.50\}$ ▷ seed(x) gives the original image of x

Group $D_{0.50}$ by mutation operator $m \in M$

for each mutation operator m **do**

$W_m \leftarrow |\{x \mid \hat{y}(x) \neq y(x)\}|$ ▷ Binary misclassification

$N_m \leftarrow |\{x \mid m(x) = m\}|$

$p_m \leftarrow \frac{W_m}{N_m}$

▷ Wilson lower-bound error estimation

$den \leftarrow 1 + \frac{z^2}{N_m}$

$center \leftarrow p_m + \frac{z^2}{2N_m}$

$adjust \leftarrow z \cdot \sqrt{\frac{p_m(1-p_m) + \frac{z^2}{4N_m}}{N_m}}$

$E_m \leftarrow \frac{center - adjust}{den}$

end for

Select the top 10 mutation operators with highest E_m:

$$M_{\text{top}} = \arg\max_{m \in M}^{10} E_m$$

Step 2: Consistency Profiling Across SSIM Thresholds $\mathcal{T}$

for each threshold $\tau \in \mathcal{T}$ **do**

$D_\tau \leftarrow \{x \in D \mid \mathrm{SSIM}(x, \mathrm{seed}(x)) \geq \tau\}$

for each mutation operator $m \in M_{\text{top}}$ **do**

$S_{m,\tau} \leftarrow \{x \in D_\tau \mid m(x) = m\}$

if $|S_{m,\tau}| > 0$ **then**

$H(m, \tau) \leftarrow \frac{|\{x \in S_{m,\tau} \mid \hat{y}(x) = y(x)\}|}{|S_{m,\tau}|} \times 100$

else

$H(m, \tau) \leftarrow \texttt{null}$

end if

end for

end for

return Consistency history matrix $H(m, \tau)$

3.5.2. Wilson error score

In the context of this work, the Wilson error score [14] refers to the lower bound of the Wilson confidence interval computed for the misclassification rate associated with a given mutation operator. Unlike raw error rates, the Wilson error score incorporates statistical confidence by accounting for the number of evaluated samples, thereby providing a robust estimate of mutation-induced error. In this work, the Wilson error score is used to rank mutation operators based on their relative effectiveness in inducing misclassification, supporting statistically grounded interpretive analysis. To quantify how strongly a mutation operator contributes to model misclassification, we compute the **Wilson score lower bound for the error rate**, as described in Algorithm 1 Given wrong predictions W, total evaluated samples $N = C + W$, and confidence constant $z = 1.96$ (corresponding to 95% confidence), this metric provides a statistically robust estimate of the

minimum error rate for each operator. Unlike a simple observed error rate, the Wilson lower bound prevents overestimating the reliability of an operator's effect when the sample size is small, ensuring that the ranking of operators is meaningful and comparable across different mutation categories. Higher values of E_w indicate greater confidence that the mutation decreases detection consistency. Therefore, mutation operators are ranked in descending order of E_w, such that the most harmful operators appear first.

The proposed analysis operates on a dataset of seed and mutated image pairs, where each sample is associated with a mutation operator, a corresponding Structural Similarity Index (SSIM) score, and a prediction outcome. SSIM is first used as a similarity-based filtering mechanism to ensure that only mutations that preserve a controlled degree of structural resemblance to the seed image are considered for analysis. At an initial baseline threshold of SSIM SSIM ≥ 0.50, the dataset was filtered to retain structurally similar mutations, which are then grouped by their respective mutation operators. For each mutation operator, the number of correctly classified and misclassified samples was computed. Rather than relying on raw error rates, a Wilson score lower bound was calculated for the misclassification rate of each operator, providing a statistically robust estimate that accounts for sample size and confidence. Mutation operators are subsequently ranked based on their Wilson error scores, and the top-ranked operators corresponding to those most likely to induce misclassification were selected for further evaluation.

To analyze the stability of mutation effectiveness across varying degrees of structural similarity, the selected mutation operators are evaluated over a range of increasing SSIM thresholds. For each threshold, the dataset is re-filtered, and the classification consistency of each selected mutation operator is computed as the proportion of correct predictions among the remaining samples. This process produces threshold-wise consistency values indexed by mutation operator, capturing how the effectiveness of individual mutations changes under progressively stricter similarity constraints. Overall, SSIM serves as a structural similarity control that governs which mutated samples are included at each stage of the analysis, while the Wilson error score provides a confidence-aware ranking mechanism to identify mutation operators that are consistently effective or ineffective. This combined analysis enables a principled interpretive assessment of mutation operator behavior and supports informed decisions for targeted dataset augmentation.

To the best of our knowledge, the combined use of SSIM for perceptual alignment and Wilson error scoring for reliability-aware ranking provides a model-agnostic and statistically grounded framework for identifying effective mutation operators, making the use of heavyweight embedding-based similarity metrics (e.g., CLIP or deep feature models) unnecessary for mutation effectiveness analysis.

4. Experimental results

In this section, we report findings and observations based on our research questions.

4.1. Discrete mutation results

The systematic application of 101 unique discrete filters has provided critical insights into the Tool (T). The results for discrete filters were processed using Tool (T)'s API via the `curl` CLI. This study underscores the importance of systematic image mutations in evaluating and enhancing the resilience of machine learning systems. We initially classified all 40 seed images from our dataset using Tool (T) to establish a baseline. Subsequently, each mutation filter was applied independently to these seed images, and Tool (T)'s predictions on the mutated images were recorded. A mutated image was labeled as a *positive* sample if it retained the same classification as its original counterpart and as a *negative* sample if its classification differed. This convention for labeling outcomes is consistently applied across all mutation categories in this study, including the continuous mutation filters (Table 2) and the grayscale-based pairwise permutations (Table 3), and is not repeated separately in those sections for brevity. Table A.6 summarizes this categorization. Notably, for the top 10 most impactful mutation filters, the consistency of Tool (T) dropped significantly to as low as 61%, as shown in Table 1, highlighting the effectiveness of these mutation filters on Tool (T).

Table 1
Top 10 mutations of discrete filters.

Mutation name	Consistency	Negative	Positive
`grayscale MS`	61%	15	24
`grayscale RMS`	65%	13	25
`flip -flip`	68%	12	26
`grayscale Average`	68%	12	26
`grayscale Brightness`	68%	12	26
`grayscale Rec601Luma`	68%	12	26
`grayscale Lightness`	71%	11	27
`grayscale Rec709Luma`	71%	11	27
`grayscale Rec709Luminance`	73%	10	28
`monochrome -monochrome`	74%	10	29

Table 2
Top 10 Mutations of continuous filters.

Mutation name	Consistency	Negative	Positive
`-black-threshold90%`	30%	27	12
`-black-threshold75%`	33%	26	13
`-black-threshold95%`	33%	26	13
`-black-threshold80%`	35%	25	14
`-black-threshold85%`	35%	25	14
`-wave40 × 40`	35%	25	14
`-wave30 × 30`	36%	24	14
`-chop80%`	37%	10	6
`-swirl-360`	39%	23	15
`-sigmoidal-contrast90090`	41%	23	16

The effect of the image categories on the origin detector with a discrete mutation filter shows variation in its results Fig. 3, consistency ranging from 80% to 93% for the top 10 affecting image categories.

Interpretation of the diagram (graph) is as follows (Refer to Fig. 3):

- Green disc – Image is classified as "*generated by T*", and it is categorized as "*safe*" by Tool (T).
- Blue disc – Image is classified as "*not generated by T*", and it is categorized as "*safe*" by Tool (T).
- Red disc – Image is classified as "*not generated by T*", and it is categorized as "*unsafe*" by Tool (T).
- Black circle – This outline circle to one of the disc shows the *ground truth* for that image i.e., the Tool (T)'s result on that image before applying any mutation filter (operator).

4.1.1. Most effective discrete mutation operators

Fig. 4 illustrates the prediction consistency trends of the ten most impactful discrete mutation operators across varying SSIM thresholds. In contrast to continuous mutations, discrete operators exhibit markedly heterogeneous and non-linear behavior. Several operators maintain high prediction consistency at lower SSIM values but experience abrupt performance degradation at intermediate thresholds, followed by partial or complete recovery as SSIM increases. This step-like behavior indicates that discrete transformations can induce abrupt changes in prediction consistency across SSIM thresholds, where small increases in structural similarity correspond to disproportionately large shifts in classifier output. Notably, some discrete operators continue to reduce prediction consistency even at high SSIM levels ($\geq$0.90). This suggests that certain discrete transformations introduce representational changes that persist despite high structural similarity, and increasing SSIM alone is insufficient to restore stable predictions.

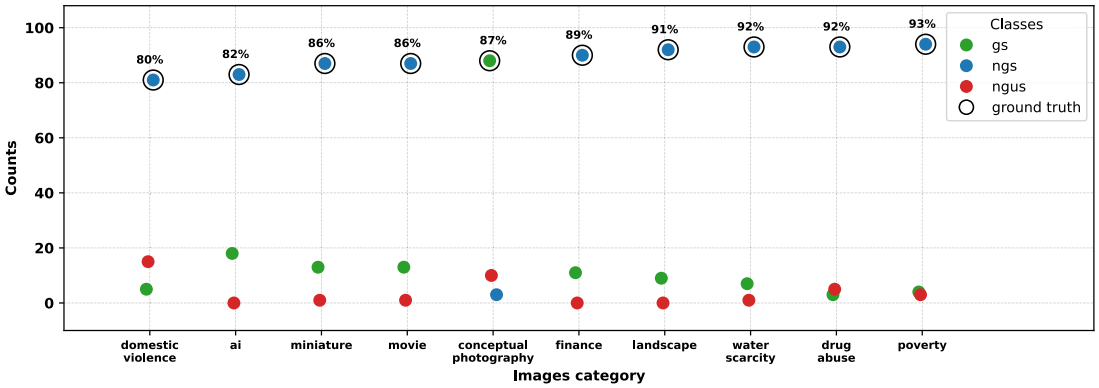


Fig. 3. Top 10 images affected by discrete filters.

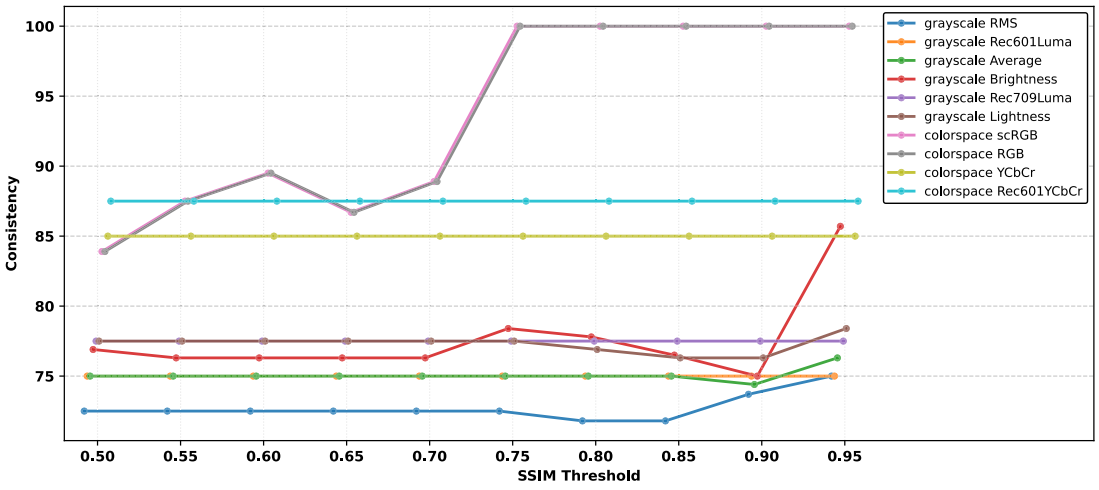


Fig. 4. Top 10 most effective mutation operators in case of discrete filter using SSIM.

4.2. Continuous mutation results

All 27 unique continuous filters were applied systematically on a dataset of 40 images to ensure consistency in the testing, which means that the testing process was uniform and repeatable in all test cases. This involves applying the same procedures, conditions, and criteria to each test to achieve reliable and comparable results. The results for continuous filters were also processed using the Tool (T)'s API via `curl` commands. The top 10 mutations of continuous filters resulted in consistently low values of classification consistency in the range of 30%–41% are shown in Table 2.

The effect of Tool (T) over different image categories with continuous mutation filter (spectrum filters) shows decrease in the Tool (T)'s performance by greater percentages as shown in Fig. 5, consistency ranging from 23% to 69% for top 10 affecting image categories.

4.2.1. Most effective continuous mutation operators

Fig. 6 shows the prediction consistency of the ten most impactful continuous mutation operators across SSIM thresholds from 0.50 to 1.00. At lower SSIM values (0.50–0.65), all selected operators cause substantial degradation, with consistency often dropping to the 50%–70% range. This indicates that continuous transformations can substantially disrupt detection consistency at lower SSIM levels, where structural similarity between the seed and mutated images is reduced. As the SSIM threshold increases (0.70–0.85), consistency improves overall; however, recovery is uneven across operators. Several mutations remain persistently less stable than others, indicating differential sensitivity to continuous transformations even when most structural content is preserved. Notably, some operators continue to induce inconsistency at high SSIM levels ($\geq$0.90), where images are visually near-identical to their originals. This demonstrates that SSIM-preserving transformations can still meaningfully affect the classifier's output.

4.3. Pair-wise permutations on grayscale filter

Permutations were applied on these filters in different orders and to the same set of images, where only two filters from a set of 9 filters are considered at a time when applying permutations. These filters were applied to each image in the dataset, and it was observed how different permutations affect the classification and safety determination. The Tool (T) was biased toward grayscale-mutated images, marking over images as "*generated by T*", and "*unsafe*", though they were not. A preliminary investigation of the Tool (T)'s consistency on two specific filters: '`grayscale MS`' and '`grayscale average`' are 61% and 68%, respectively, as shown in Table 3 & Table 1. When the filters are applied together in different orders, we observe a pronounced deviation in the outcome of the Tool (T).

Pair-wise filter mutations lead to more robust images for testing against the Tool (T), which proves its effectiveness (Fig. 7) by decreasing the maximum consistency of the Tool (T) for top 10 image categories to only 41%, with the least consistency going to 0% for the image "conceptual photography".

4.3.1. Pair-wise based mutations—Most effective operators

Fig. 8 shows the prediction consistency of the ten most effective permutation-based mutation operators. Among all mutation families,

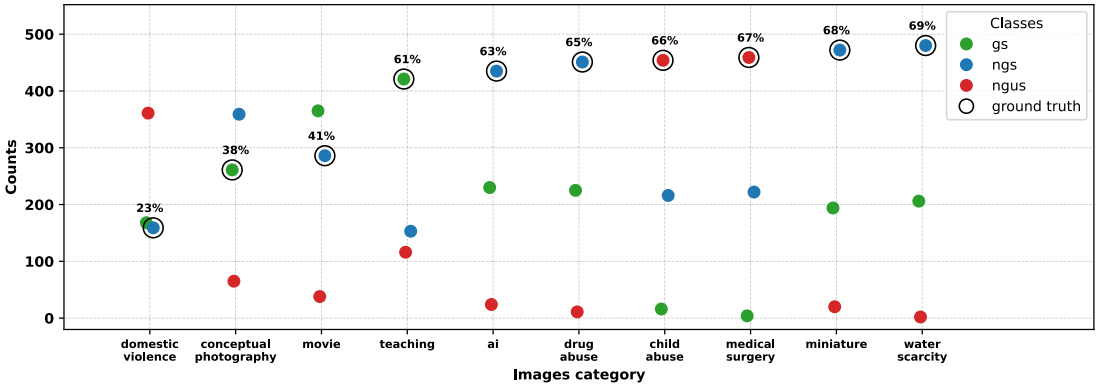


Fig. 5. Top 10 images affected by continuous filters.

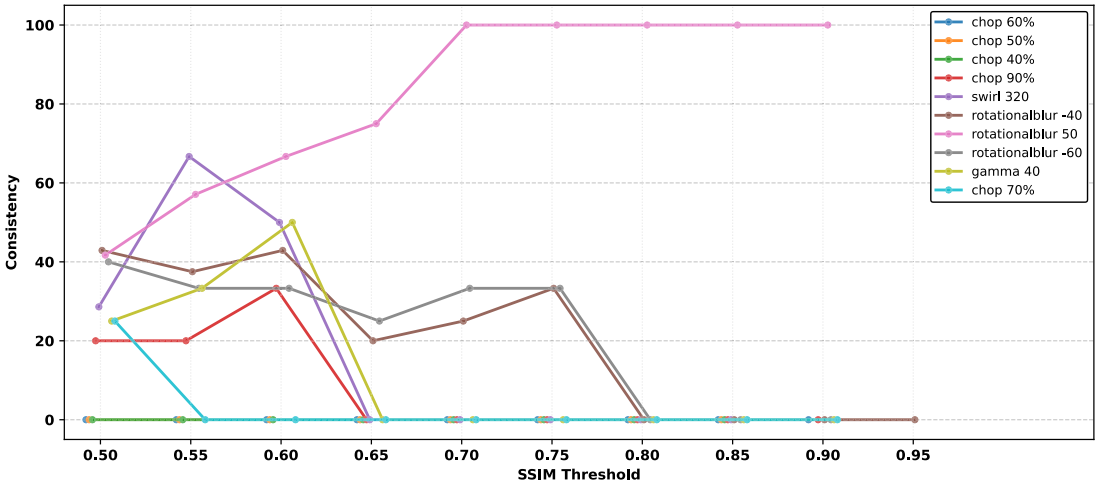


Fig. 6. Top 10 most effective mutation operators in case of continuous filter using SSIM.

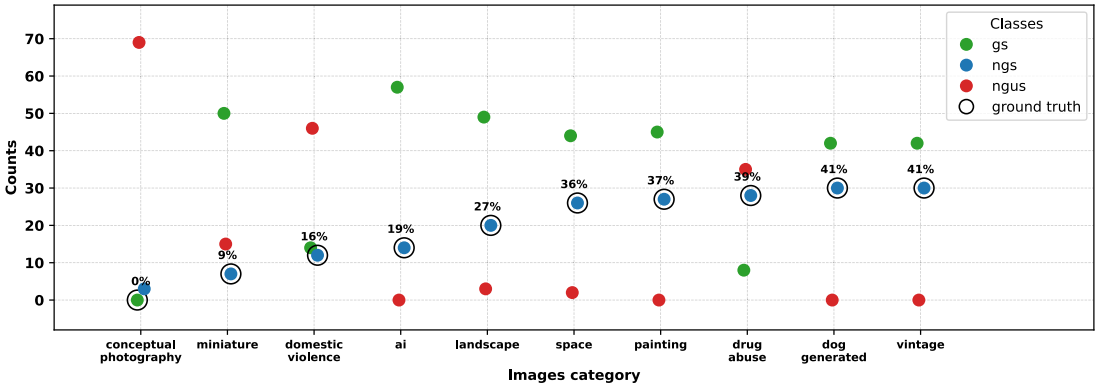


Fig. 7. Top 10 images affected by pair-wise filters permutations.

permutation operators exhibit the most severe and persistent degradation across the SSIM range. Multiple operators maintain low consistency levels from low to high SSIM values, with only marginal recovery as SSIM approaches 1.00. This indicates that permutation-based transformations can lead to persistent prediction instability even when SSIM values remain high. The limited improvement in prediction consistency with increasing SSIM suggests that high structural similarity alone is insufficient to ensure robust and stable classifier behavior under these transformations.

4.4. Least-effective mutation operators

A subset of mutation operators exhibits consistently high prediction consistency across all evaluated SSIM thresholds, indicating minimal

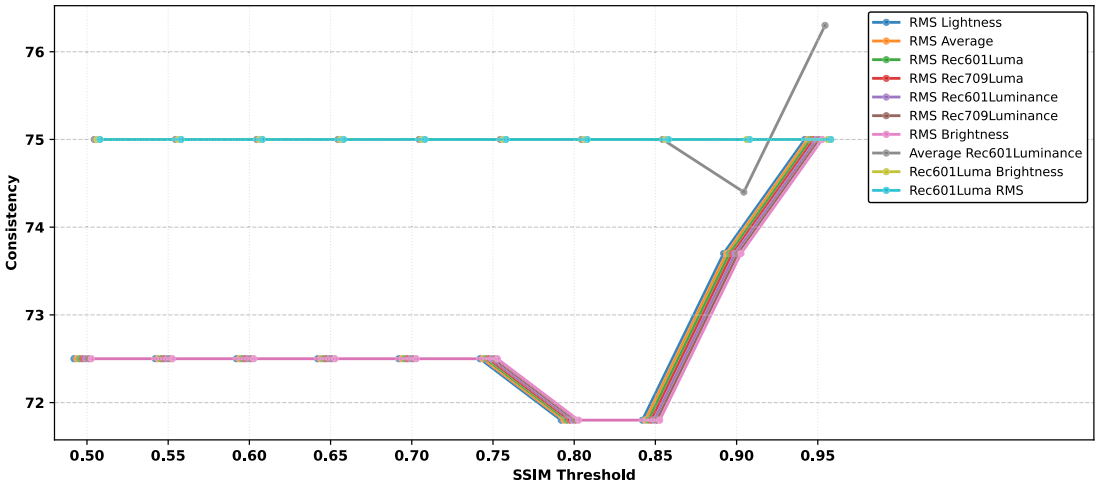


Fig. 8. Top 10 most effective mutation operators in case of pair-wise permutation filter using SSIM.

Table 3
Top 10 mutations of pair-wise permutation of grayscale filter.

Mutation name	Consistency	Negative	Positive
Average MS	51%	19	20
Rec601Luma MS	51%	19	20
Rec709Luminance MS	53%	18	21
Lightness MS	56%	17	22
Rec601Luminance MS	58%	16	23
MS Average	61%	15	24
MS Brightness	61%	15	24
MS Lightness	61%	15	24
MS RMS	61%	15	24
MS Rec601Luma	61%	15	24

influence on the classifier's output. The 10 least-effective operators, spanning discrete, continuous, and pair-wise permutation categories, are ranked according to their Wilson error scores and demonstrate uniformly high consistency that remains effectively constant across the full SSIM range. Within the discrete category, the least-effective operators are dominated by color-space conversions (e.g., `YDbDr`, `Jzazbz`, `HSV`, `LMS`, `xyY`) and ICC profile transformations (e.g., `CGATS21CRPC1`, `ColorMatchRGB`). The continuous operators ranked as least effective primarily consist of low-intensity spatial or contrast modifications, including mild blurring (`blur 100x0`, `gaussianblur 70x0`), contrast adjustments (`sigmoidalcontrast` variants), and non-disruptive geometric edits (`border 0%`, `chop 100x0`, `splice 40%×40`). For pair-wise permutation mutations, the least-effective operators are predominantly averaging-based or luminance-preserving combinations (e.g., `MS Average`, `Rec709Luma MS`, `Rec601Luma MS`, `MS Brightness`), which maintain stable prediction behavior across all SSIM thresholds. As summarized in Table 4, the linear and stable behavior of these operators justifies omitting detailed SSIM-trend plots, as the tabulated results sufficiently capture their negligible impact on robustness.

4.5. Image scaling

The image down-scaling process involved reducing the dimensions of each image (height and width) by 10% in each iteration from the existing seed image resolution; it was reduced continuously until the image reached a minimum size of 1 pixel, at each step of size reduction, a mutated image gets generated, which is then used for analysis using `curl` CLI (API call). Similarly, the up-scaling process in which each image size was gradually increased by 10%, and the expansion continued until the image reached a maximum resolution of 10,000 pixels. These mutated images were used for further analysis and testing of the Tool (T). This approach allowed us to track how the Tool (T)'s performance varied with image resolution and determine the Tool (T)'s sensitivity to resolution reductions. The goal was to examine how the Tool (T) responds to the resizing of image resolution, particularly in conditions where the images undergo significant upscaling. Up/Down(scaling) operations were applied on the dataset of 40 images produced a total of 2773 images compared to the seed image's resolution. Our findings indicate that the minimum resolution of the scaled seed image is ∼280 pixels (width or height).

Image scaling worked as a tool to validate the Tool (T)'s potential, and to find its optimal operable image size. As shown in Fig. 9, Tool (T) consistency predominantly hovers around 50%. For each image, the consistency varies, ranging from a minimum of 24% to a maximum of 50% for the top 10 image categories.

4.6. Answering RQ1 (mutation operators)

Motivation: Existing literature has laid the foundation for this work by exploring basic image mutation techniques for data augmentation, such as rotation, cropping, scaling, blurring, and color tone adjustments. However, since AI models typically learn to handle these mutations during training, they may become vulnerable to real-world variations not seen during augmentation. This reinforces our motivation to evaluate the Tool (T) using a more exhaustive and diverse set of image mutations.

Approach: To minimize manual effort and streamline the repetitive process, we automated the entire pipeline using multiple Python scripts—from generating mutated images to feeding them into the Tool (T) for evaluation.

Results: As shown in Fig. 3, the Tool (T)'s consistency dropped from the claimed value of over 90% to approximately 80%. In addition, certain mutation filters, listed in Table 1, reduced the consistency even further, down to around 61%.

> Application of 101 unique discrete filters (systematic mutations) to a dataset of 40 images produced 4040 mutated images. The application of 27 unique continuous filters (systematic mutations) to a dataset of 40 images produced a total of 27,640 mutated images, with each image subjected to 691 mutations. Application of up/down-scaling operators on the dataset of 40 images produced a total of 2773 images with dimensions in the range of 1 to 10,000 pixels (width or height) in steps of ±10% compared to the seed image's resolution.

Table 4
Ranked top-10 least-effective image mutation operators by category.

Rank	Discrete operators	Continuous operators	Pair-wise permutation operators
1	colorspace YDbDr	wave 0 × 90	MS Average
2	colorspace Jzazbz	+sigmoidalcontrast 0 × 60	Lightness MS
3	profile CGATS21CRPC1	raise 0	Average MS
4	colorspace HSV	splice 40%x40	Rec709Luma MS
5	colorspace LMS	blur 100 × 0	MS Rec709Luma
6	colorspace YPbPr	gaussianblur 70 × 0	Rec601Luminance MS
7	profile ColorMatchRGB	border 0%	Rec601Luma MS
8	colorspace xyY	chop 100 × 0	MS RMS
9	profile GRACoL2006Coated1v2	+sigmoidalcontrast 0 × 90	MS Rec709Luminance
10	profile JapanColor2002Newspaper	sigmoidalcontrast 0 × 70	MS Brightness

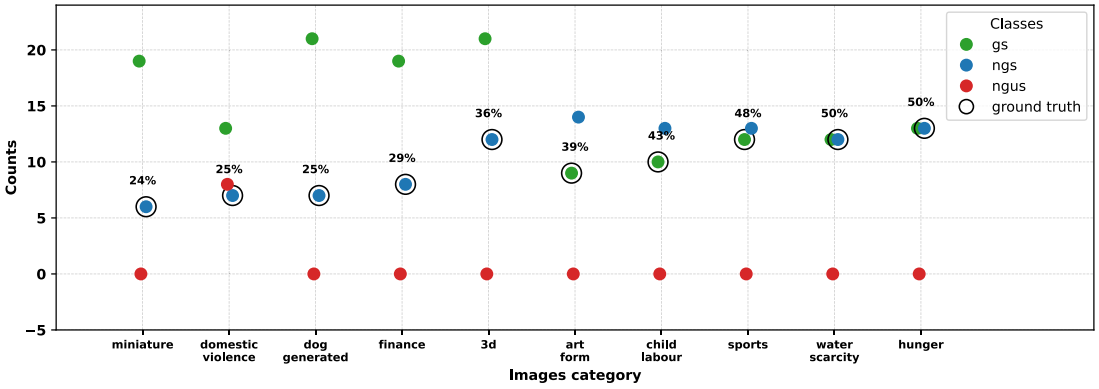


Fig. 9. Top 10 images affected by image resizing.

4.7. Answering RQ2 (mutation spectrum)

Motivation: Spectrum mutation filters refer to parameterized transformations that apply a range of effects within a single filter, using either integer or floating-point values. Findings from RQ1 suggest that exposing the Tool (T) to more exhaustive and varied mutations may result in a more significant decline in classification performance.

Approach: Each mutation filter offered a large number of parameter variations — for example, the RGB color filter alone had approximately 1.6 million possible combinations — which made exhaustive testing impractical. Therefore, for this study, we selected a subset of parameter values either at regular intervals or through random sampling.

Results: Consistent with the trend observed in the results of RQ1, spectrum mutation filters revealed additional vulnerabilities in the Tool (T), as shown in Fig. 5, with classification consistency dropping to between 23% and 69%. Moreover, results from Table 2 shows robust filters that leads to classification consistency to 30%.

> The use of discrete filters brought down the Tool (T)'s consistency to as low as 61%. This suggests further analyses of different mutation categories. The top 10 continuous filters resulted in consistently low values of classification consistency in the range of 30%–41%. The Tool (T)'s output can change across different dimensions (scaling) of the same seed image, provided the minimum resolution of the scaled seed image is ~280 pixels (width or height).

4.8. Answering RQ3 (mutation combinatorics)

Motivation: Pairwise permutations of mutation filters — such as crop followed by rotate, rotate followed by blur, or crop followed by blur — represent a basic approach to image mutation. However, these permutations typically involve only a limited set of mutation operators, leaving significant room for further exploration. Our observations reveal that image mutations are not commutative; that is, applying filter B after filter A yields a different result than applying filter A after filter B. This non-commutative property of mutation filters serves as the primary motivation for our study.

Approach: We selected 9 `grayscale` mutation filters for this study and generated all possible ordered pairs, resulting in 72 unique combinations. These filter combinations were applied to each image using automated Python scripts.

Results: This experiment led to a more significant drop in Tool (T)'s consistency compared to spectrum-based mutation filters, with consistency falling as low as 0% and reaching a maximum of only 41% across the top 10 categories as shown in Fig. 7 and Table 3 shows the consistency of the pair-wise filters permutations. Moreover, the combined results from all three research questions (RQs) reinforce our hypothesis regarding existing vulnerabilities in the training and testing pipelines of AI models.

> A preliminary investigation of the Tool (T)'s consistency on two specific filters: "`grayscale MS`" and "`grayscale average`" are 61% and 68%, individually. When the filters are applied together in different orders, we observe a pronounced deviation in the Tool (T)'s outcome.

4.9. Diagnostic insights into mutation-induced failures

In the absence of access to the internal representations of the evaluated black-box classifier, the precise mechanisms underlying prediction failures cannot be directly determined. Nevertheless, consistent

trends observed across mutation families allow for informed interpretation supported by prior literature. Color and grayscale-based mutations consistently emerged among the most impactful operators. Prior studies have shown that classifiers trained primarily on RGB representations often rely on chromatic cues that are degraded or lost under grayscale or color-space transformations, leading to reduced unstable predictions [19]. This suggests that limited invariance to color representation contributes to the observed prediction inconsistency. Geometric transformations, such as rotation, produced sustained instability across thresholds. Existing work demonstrates that convolutional architectures are not inherently rotation-invariant and can exhibit sharp performance degradation when spatial alignment assumptions are violated [20,21]. These findings help explain why increasing similarity alone does not reliably restore prediction stability under such transformations. Resolution-related and blur like continuous mutations further affected robustness. Prior robustness benchmarks indicate that models are particularly sensitive to degradations that alter fine-scale texture, even when overall image structure appears preserved [22,23]. This sensitivity likely contributes to the persistent inconsistency observed for several continuous operators. Permutation-based mutations produced the most sustained degradation. While local image content may remain unchanged, prior work shows that deep models encode strong implicit assumptions about feature arrangement, and violations of these assumptions can invalidate learned representations [24,25]. This explains why certain permutation operators remain harmful regardless of similarity magnitude. From a mitigation perspective, these observations highlight the importance of incorporating mutation operators that induce persistent or abrupt instability into robustness evaluation pipelines. Prior work suggests that training with structurally diverse transformations can improve tolerance to such perturbations [26], indicating a potential direction for strengthening image-origin detection systems against structured input variations.

4.10. Limits of generalizability and adaptability under SSIM-preserving transformations

4.10.1. Generalizability

The results show that robustness does not generalize uniformly across mutation operators, even under similar SSIM conditions. Certain mutations consistently destabilize predictions across a wide SSIM range, while others remain stable throughout. This indicates that robustness cannot be reliably inferred from structural similarity alone and that evaluations limited to a narrow set of mutations may overestimate generalization.

4.10.2. Adaptability

The analysis reveals limited adaptability of the classifier to several SSIM-preserving transformations. For the most effective mutations, increasing SSIM does not consistently restore prediction stability, whereas the least effective mutations maintain high consistency across all thresholds. This suggests that adaptability depends on the specific transformation rather than the degree of structural similarity.

5. Downstream usage

The primary audience for this work includes researchers, software testers, and system developers engaged in the development and evaluation of AI models that consume digital images as inputs. While our mutation-based strategy was originally designed to evaluate the robustness of commercial image origin classifiers, the methodology generalizes well to a broader class of image-processing AI systems. Our approach is particularly applicable to the robustness testing of multimodal AI systems, including Large Language Models (LLMs) that process images as part of their input. It can serve as a foundational tool for probing the failure modes of such models by introducing controlled, systematic image-level perturbations.

Concrete downstream applications of our method span several image-based AI tasks.

- In image captioning systems (image-to-text), small visual perturbations can lead to semantically inaccurate or misleading textual descriptions, highlighting the need to evaluate input sensitivity.
- In image-to-image translation tasks—such as photorealistic enhancement or converting images across domains (e.g., day-to-night or sketch-to-photo)—even minor input alterations can impact the output, making systematic evaluation essential.
- Similarly, object detection systems, especially those deployed in real-time applications like autonomous driving systems (ADS) or unmanned aerial vehicles (UAVs), must maintain reliable performance under diverse conditions such as varying lighting, partial visibility of objects, and changes in viewpoint or orientation.

In safety-critical domains such as transportation, healthcare imaging, surveillance, and aerial monitoring, our methodology provides a reproducible way to generate mutation datasets for validating the reliability and trustworthiness of AI systems under controlled perturbations. Given its generality, scalability (it can be applied to different types of AI systems and datasets), and black-box applicability (it can be used without detailed knowledge of the AI model's internal workings), the proposed mutation-driven testing methodology enables controlled experimentation to evaluate the behavior and reliability of image-based models across diverse visual processing tasks.

6. Related work

In this section, we discuss relevant prior work: (i) Image augmentation, (ii) Image mutation, and (iii) Image classification.

6.1. Image augmentation

Image data augmentation is a trivial technique for enhancing the performance and robustness of deep learning models, especially in image classification tasks [2]. Since deep models typically require large datasets to generalize well, their performance can degrade under adversarial conditions—where imperceptible input perturbations cause incorrect predictions [27–32]. To address this vulnerability and improve model generalization, data augmentation generates modified instances of a seed image to expand training diversity.

Traditional augmentation methods [33] include geometric transformations (e.g., rotation, flipping, scaling) that preserve an image's core structure while altering its orientation [34]. Additionally, non-geometric techniques such as color adjustments, cropping, and resizing introduce further variation. As reviewed in Kumar et al. [2], these transformations can be categorized under a well-defined taxonomy. While that work primarily focuses on augmentations for model training, we adopt the broader term *mutations* to denote a wider set of image modifications applicable to black-box testing and evaluation.

6.2. Image mutation

Although traditional augmentation techniques are effective, they have limitations. Operations like cropping or rotating small-resolution images may eliminate essential features or degrade image quality, thereby reducing classifier consistency. To overcome these drawbacks, image mutation methods aim to produce more diverse and complex variants. Generative Adversarial Networks (GANs) are one such approach, capable of generating new images for data augmentation. However, their performance is often constrained by the size and quality of the training dataset [35,36]. Recently, Multi-modal Large Language Models (MLLMs), such as GPT-4V [37], have emerged as powerful tools for mutating images based on natural language prompts. By integrating diffusion models [38] with advanced semantic understanding, MLLMs

can perform complex and context-aware image modifications beyond the capabilities of traditional methods. Nevertheless, interpreting the internal generative processes of such models remains challenging [39–44]. Pei et al. [9] presented a gradient-based optimization method for pixel to pixel mutation. The proposed method of image up/down scaling is inspired by [10]. Furthermore, recent studies highlight the instability of generative outputs, which can compromise testing reliability [45–47]. Our work extends this body of research by exploring a broader range of image mutation methods than those covered in [2,48], particularly in the context of black-box evaluation. Additionally, we draw inspiration from the fine-grained rotational mutation strategies proposed in [49] to rotate images with minute degrees as part of our ongoing work.

6.3. Image classification

Image classification is used in many areas of machine learning; the primary task is to assign a label or category to an image based on its contents. It can be used in medical imaging, AI and fake image detection. The basic idea is to train a machine learning model like DNN on a dataset of labeled images, and the model then learns the features associated with an image. Once trained, it can predict the class of an unseen image based on its learning. The popular image classification datasets are CIFAR-10/CIFAR-100 [50], ImageNet [51], and MNIST, which are single-label and each image can be classified into one class. Convolutional neural networks (CNNs) [52], which are a type of Deep neural networks (DNNs), are used for image classification. While classifier we used was evaluated based on the results received on API calls made on mutated images. we noticed that the model consistency remained in the range of 30%–41% when the seed image was mutated using a continuous filter (mutation), and the model misclassified the unsafe image depicting domestic violence as safe.

7. Threats to validity

7.1. Internal validity

This assesses whether the results truly reflect the intended phenomenon or are influenced by unintended factors. The Python pipeline or JSON parsing might introduce unnoticed preprocessing bugs or incorrect result mappings, affecting result integrity. Using `ImageMagick`, we assume uniform behavior across all mutations, but undocumented behavior in command execution or OS-level rendering differences may introduce unintended artifacts. If the image origin classifier retains session-based state or learns over time (e.g., via training), repeated tests might not be independent, biasing results.

7.2. Construct validity

This assesses whether we are measuring the image origin classifier's performance as claimed. The evaluation in this study is grounded on a dataset of 40 seed images, each assumed to be unambiguous and correctly classifiable. Any ambiguity in origin or latent artifacts within these images could undermine the reliability of the ground truth and, by extension, the validity of the classifier's performance assessment. As the evaluated system is a proprietary black-box classifier and independent ground-truth labels are unavailable, this study measures prediction consistency relative to the model's baseline output rather than correctness with respect to true image origin. Accordingly, reported consistency values should be interpreted as indicators of robustness under mutation, not absolute detection consistency. While the classifier's outputs—returned in structured formats such as `JSON`—are ultimately mapped to binary correctness labels (e.g., "right" or "wrong"), this simplification may obscure nuanced behaviors of the model. Although this abstraction is necessary for quantitative comparison, it is recognized as a limitation in fully capturing the complexity of the classifier's operations.

7.3. External validity

This assesses whether our results generalize to other classifiers. The 40 original images, while sufficient for mutation scale, may not represent the diversity (e.g., camera sources, lighting, content type) seen in operational environments. Results might not generalize to news media, surveillance, or social media imagery, as we have not covered all the categories of images. The evaluated tool is closed-source and undergoes continuous retraining across different releases. Since our experiments were conducted on a single available version, the results may not generalize to past or future versions of the tool and pertain to a specific version of classifier that we have used. Other tools (e.g., forensic detectors, camera fingerprint-based methods) might behave differently under the same mutations. If the classifier is updated over time, results may only reflect its behavior during a specific deployment window and not future iterations.

7.4. Conclusion validity

This assesses whether our inferences are logically sound and statistically valid. Without evaluating classifier performance on unmutated images as a reference in each mutation batch, some degradation trends may be misattributed, i.e., if we do not check how well the classifier works on the original (unchanged) images each time, we might wrongly guess what is causing its performance to drop. We have treated mutation categories uniformly, but some transformations may be over- or under-represented in terms of image impact. If human labels were used to verify classifier correctness, inter-rater agreement or annotator bias could be a concern. If not used, any mismatch between the tool output and the ground truth may go unchallenged.

8. Future scope

In this study, we found that the classifier correctly identified unmodified images, but it had difficulty in accurately identifying mutated images, especially those marked as unsafe (like those showing domestic violence). In a few of the cases, when these images were rotated by certain angles, the classifier wrongly labeled them as safe, even though they were not. Future work may focus on rotating images across a 0°–360° range to evaluate how these rotations impact the classifier's ability to detect unsafe content. These rotated images will be compared to the original ones, with particular attention given to unsafe images where rotation caused false negatives in earlier tests. Understanding how changes in image orientation influence classification consistency will be a key area of focus. Additionally, further experiments may explore combining rotations with other transformations, such as grayscale permutations, to see how different filter combinations affect the classifier's performance. An important part of this study will be to investigate whether the rotation of images could lead to inaccurate safety assessments, including potential false positives or missed unsafe content. As part of ongoing efforts, more tests could involve pairwise permutations alongside rotations and additional filters to refine the classifier's handling of image variations.

The development of a semi-autonomous tool for similar testing is at the core of this study. Future work will comprise creating a user-friendly tool with minimal efforts and active engagement time to do similar tests and validation, providing a comprehensive report of the experiment. Investigating the impact of file format variations on classifier outputs offers an opportunity for a deeper assessment of model sensitivity to variances in input encoding. This framework can be extended to multiple image-origin classifiers to facilitate comparative robustness analysis under identical mutation and perceptual similarity

constraints. Repeating mutation experiments across multiple runs to estimate confidence intervals and effect sizes can further enhance statistical transparency in robustness evaluation. Future work will include evaluating the framework on larger and benchmark image datasets to strengthen generalizability of the observed robustness patterns.

9. Conclusion

In this study, we found ways to mutate images and use them as input to test the classifier's performance. As reflected in our experimental results, we were able to perform robustness testing of the classifier and found that it was biased on grayscale mutations. The discrete mutations brought down the classifier's consistency to as low as 61%, and continuous mutations resulted in a range of 30%–41% consistency. It was found that different dimensions of the same image could lead to different results, and the minimum acceptable limit of image resolution was found to be ≈280 width or height.

CRediT authorship contribution statement

Vaishnav Koka: Writing – original draft, Visualization, Validation, Software, Methodology, Formal analysis, Data curation, Conceptualization. **Ramanand:** Writing – original draft, Visualization, Validation, Software, Methodology, Formal analysis, Data curation, Conceptualization. **Shouvick Mondal:** Writing – original draft, Writing – review & editing, Supervision, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis. **Yogesh Kumar Meena:** Writing – original draft, Writing – review & editing, Supervision, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used `grok`, and `perplexity` in order to enhance writing (some parts of the manuscript). After using these tools/services, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

Funding sources

This work was supported by IIT Gandhinagar (Grant Nos. IP/IITGN/CSE/SM/2324/02 and IP/IITGN/CSE/YM/2324/05).

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Shouvick Mondal reports financial support was provided by Indian Institute of Technology Gandhinagar. Yogesh Kumar Meena reports financial support was provided by Indian Institute of Technology Gandhinagar. Shouvick Mondal reports a relationship with Indian Institute of Technology Gandhinagar that includes: employment and funding grants. Yogesh Kumar Meena reports a relationship with Indian Institute of Technology Gandhinagar that includes: employment and funding grants. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix

See Table A.5

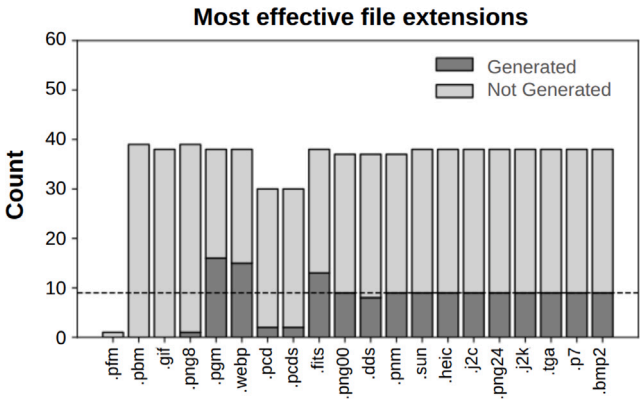


Fig. A.10. Effect of image file extensions on Tool (T)'s classification.

A.1. Image extensions

To evaluate the robustness of Tool (T), its behavior was assessed across **157 distinct image file extensions**, each generated from a fixed seed image. The objective was to assess whether the Tool (T)'s output varies solely due to a change in the file format. Analysis revealed that both the *Transformation* and *Confidence* values produced by Tool (T) were indeed sensitive to the file extension applied. For clarity and relevance, only the most impactful results are presented in this section.

Interpretation of the diagram (graph) is as follows (Refer to Fig. A.10):

- Dark Gray – Tool (T) classified the image as "*generated*" by AI Tool
- Light Gray – Tool (T) classified the image as "*not generated*" by AI Tool

In particular, Tool (T) failed to return a decision for some formats (e.g., `.pfm`), while others exhibited consistent yet biased behavior. For instance, images saved as `.pbm` were consistently classified as "*generated*" regardless of their visual content, indicating a potential systemic bias. Similar trends were observed for extensions such as `.png8`, `.pcd`, `.pcds`, and `.dds`, which skewed heavily toward the "*not generated*" classification. Conversely, formats like `.pgm`, `.webp`, and `.fits` exhibited a bias toward the "*generated*" class. Meanwhile, the remaining extensions produced classification results identical to the original image set, suggesting negligible impact from those format changes. Based on these findings, we identify `.pbm`, `.gif`, `.png8`, `.pgm`, `.webp`, `.pcd`, `.pcds`, `.fits`, and `.dds` as particularly effective extensions for probing the classifier's sensitivity to file format variation.

A.2. Supplementary results for least-effective mutations

A.2.1. Least effective discrete mutation operators

Fig. A.11 presents the prediction consistency trends for the ten least effective discrete mutation operators across SSIM thresholds. These operators demonstrate uniformly high consistency throughout the full SSIM range, with no abrupt transitions or declines observed at any threshold. Unlike the most effective discrete mutations, increasing or decreasing SSIM has minimal impact on detection consistency for this subset and as SSIM increases, consistency remains same, reflecting that performance is already near optimal. The stable behavior across all thresholds indicates that these discrete transformations do not introduce changes that significantly influence classifier outputs, even under reduced structural similarity.

Table A.5
Dataset of 40 seed images with linked references.

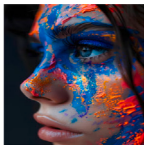
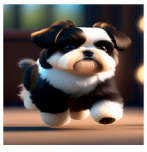
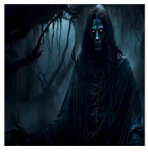
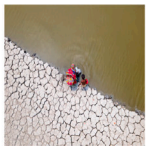
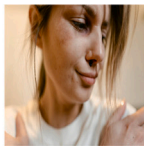
Dataset				
 (1) 3d	 (2) abstract_design	 (3) adventure	 (4) aerial_view	 (5) ai
 (6) animals	 (7) anime_cartoon	 (8) art_form	 (9) child_abuse	 (10) child_labour
 (11) city	 (12) con_photography	Image Not Available (13) Img_1 [11]	Image Not Available (14) Img_2 [11]	Image Not Available (15) Img_3 [11]
 (16) dog_generated	 (17) domestic_abuse	 (18) domestic_violence	 (19) drug_abuse	 (20) drugs
 (21) festival	 (22) finance	 (23) food	 (24) ghost	 (25) hunger
 (26) landscape	 (27) medical_surgery	 (28) miniature	 (29) movie	 (30) painting
 (31) poverty	 (32) riot	 (33) space	 (34) sports	 (35) teaching
 (36) toy	 (37) vintage	 (38) war	 (39) water_scarcity	 (40) women_violence

Table A.6
Representative set of image mutation operators (command-line flags) in `ImageMagick 7.1.1-41` and their descriptions, showing one instance per filter type due to space constraints.

S.No	Generic flag	Specific flag	Outcome
		Discrete Mutation Operators	
1	`-colorspace`	Convert to CMY Colorspace *Ex: magick <input.jpg> -colorspace CMY <output.jpg>*	Converts the image to the CMY (Cyan, Magenta, Yellow) colorspace, often used in printing where these three colors are combined to create various hues.
2	`-grayscale`	Grayscale Conversion using Rec601 Luminance *Ex: magick <input.jpg> -grayscale Rec601Luminance <output.jpg>*	Converts the image to grayscale using the Rec. 601 Luminance formula (linear RGB).
3	`-profile filename`	sRGB_v4_ICC_preference_displayclass *Ex: magick <input.jpg> -profile sRGB_v4_ICC_preference_displayclass.icc <output.jpg>*	Applies the sRGB v4 ICC preference display class color profile to the image.
4	`-flip`	Basic Flip *Ex: magick <input.jpg> -flip <output.jpg>*	Flips the image vertically (top to bottom).
5	`-flop`	Basic Flop *Ex: magick <input.jpg> -flop <output.jpg>*	Flops the image horizontally (left to right).
6	`-contrast`	Increase Contrast by One Level *Ex: magick <input.jpg> -contrast <output.jpg>*	Increases the contrast of the image by one level, making colors more vivid.
7	`-monochrome`	Basic Monochrome *Ex: magick <input.jpg> -monochrome <output.jpg>*	Converts the image to monochrome, applying a black-and-white threshold.
8	`-negate`	Basic Negate *Ex: magick <input.jpg> -negate <output.jpg>*	Inverts the colors of the image, turning black to white and vice versa.
9	`-normalize`	Basic Normalize *Ex: magick <input.jpg> -normalize <output.jpg>*	Normalizes the image by scaling pixel values to the full range of colors.
		Continuous Mutation Operators	
10	`-colors`	Reduce Image to 16 Colors *Ex: magick <input.jpg> -colors 16 <output.jpg>*	Reduces the number of colors in the image to 16, creating a simplified, posterized version with fewer shades. This can create a more abstract or "flat" look.
11	`-edge`	Basic Edge Detection *Ex: magick <input.jpg> -edge 1 <output.jpg>*	Detects edges in the image using a 1-pixel radius.
12	`-emboss`	Basic Emboss *Ex: magick <input.jpg> -emboss 1 <output.jpg>*	Applies a basic emboss effect to the image with a radius of 1.
13	`-statistic Median`	Basic Median Filter *Ex: magick <input.jpg> -statistic Median 2 <output.jpg>*	Applies a median filter with a 2-pixel radius, reducing noise while preserving edges.
14	`-paint`	Basic Paint (Level 1) *Ex: magick <input.jpg> -paint 1 <output.jpg>*	Applies a paint effect using a 1-pixel brush.
15	`-posterize`	Posterize Level 2 *Ex: magick <input.jpg> -posterize 2 <output.jpg>*	Reduces the image to 2 levels of color.
16	`-swirl`	No Change *Ex: magick <input.png> -swirl 0 <output.png>*	The original image remains intact.
17	`-spread`	Minor Pixel Displacement *Ex: magick <input.jpg> -spread 2 <output.jpg>*	Slight degradation in smooth textures or edges.
18	`-rotate`	Rotate by 90 Degrees *Ex: magick <input.jpg> -rotate 90 <output.jpg>*	Rotates the image 90 degrees clockwise.

(*continued on next page*)

Table A.6 (*continued*).

19	-charcoal	Basic Charcoal Effect *Ex: magick <input.jpg> -charcoal 1 <output.jpg>*	Applies a charcoal drawing effect with a radius and sigma of 1, creating a light sketch-like appearance with soft edges.
20	-rotational -blur	Basic Rotational Blur *Ex: magick <input.jpg> -rotational-blur 5 <output.jpg>*	Applies a rotational blur with a radius of 5 pixels.
21	-gamma	Basic Gamma Correction *Ex: magick <input.jpg> -gamma 2.2 <output.jpg>*	Adjusts the image gamma to 2.2, brightening it overall.
22	-implode	Basic Implode *Ex: magick <input.jpg> -implode 0.5 <output.jpg>*	Reduces the image size by 50% using implode effect.
23	-solarize	No pixels are inverted; the image remains unchanged. *Ex: magick <input.jpg> -solarize 0% <output.jpg>*	Original image without any effect.
24	-white-balance	Complete Desaturation *Ex: magick <input.jpg> -define white-balance:vibrance=-100% -white-balance <output.jpg>*	Complete desaturation.
25	-threshold	All Pixels Become White *Ex: magick <input.jpg> -threshold 0% <output.jpg>*	Since every pixel value is considered ≥0% threshold, all are set to maximum intensity (white).
26	-black -threshold	Simple Black Threshold (with Percentage) *Ex: magick <input.jpg> -black-threshold 25% <output.jpg>*	Any pixel value less than 25% brightness will be set to black, and values above will be unchanged.
27	-tint	Barely Noticeable Tint *Ex: magick <input.jpg> -fill blue -tint 10% <output.jpg>*	Original image almost unchanged, with slight hints of fill color.
28	-wave	No Wave Effect *Ex: magick <input.png> -wave 0 × 20 <output.jpg>*	The image remains unchanged.
29	-wavelength	Very Tight Waves *Ex: magick <input.png> -wave 10 × 10 <output.jpg>*	Waves appear very close together; tight sine wave effect.
30	-blur	Simple Blur with Radius and Sigma *Ex: magick <input.jpg> -blur 0 × 5 <output.jpg>*	This applies a Gaussian blur with a sigma of 5, resulting in a moderate blur across the entire image. The radius is automatically computed based on the sigma.
31	-border	Simple Border with Default Color *Ex: magick <input.jpg> -border 10 × 10 <output.jpg>*	Adds a 10-pixel wide border on all sides of the image using the default color (usually black). The image will have equal borders around it.
32	-chop	Simple Horizontal Chop *Ex: magick <input.jpg> -chop 100 × 0 <output.jpg>*	Removes 100 pixels from the width of the image, effectively chopping off 100 pixels from the left or right side (depending on gravity settings), leaving the height unchanged.
33	-sigmoidal -contrast	No contrast change; image remains unchanged. *Ex: magick <input.jpg> -sigmoidal-contrast 0 × 50 <output.jpg>*	Original image without effect.
34	-gaussian-blur	Basic Gaussian Blur *Ex: magick <input.jpg> -gaussian-blur 0 × 2 <output.jpg>*	Applies a Gaussian blur with a radius of 0 and a sigma of 2.
35	-splice	Adds 20 pixels along horizontal edges. *Ex: magick <input.jpg> -background blue -splice 20 × 0 <output.jpg>*	Narrow vertical padding on both sides.
36	-raise	Raise Image by 10 pixels *Ex: magick <input.jpg> -raise 10 <output.jpg>*	Raises the image by 10 pixels.

For the complete list of filters and their variations, refer to: https://osf.io/fau7k.

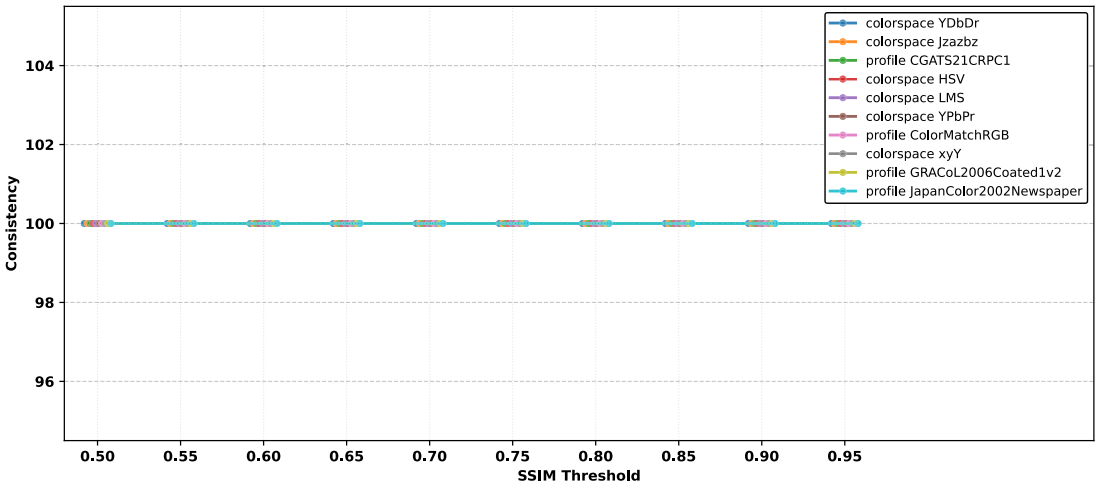


Fig. A.11. Top 10 least effective mutation operators in case of discrete filter using SSIM.

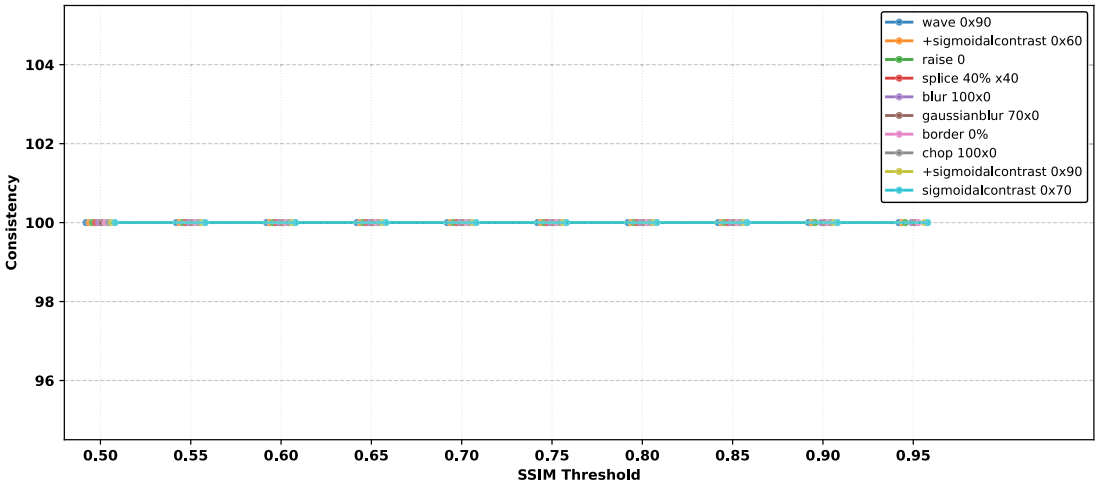


Fig. A.12. Top 10 least effective mutation operators in case of continuous filter using SSIM.

A.2.2. Least effective continuous mutation operators

Fig. A.12 illustrates the prediction consistency of the ten least effective continuous mutation operators across SSIM thresholds from 0.50 to 1.00. In contrast to the most effective operators, these mutations exhibit consistently high prediction consistency across the entire SSIM range. No noticeable degradation is observed at lower SSIM thresholds, and consistency remains stable as SSIM increases. The absence of performance degradation indicates that these continuous mutations do not meaningfully affect the classifier's predictions, even when structural similarity between original and mutated images is reduced. Overall, the results suggest that these operators preserve the characteristics required for stable classification across all evaluated SSIM levels.

A.2.3. Pair-wise based mutations—Least effective operators

Fig. A.13 shows the prediction consistency of the ten least effective permutation-based mutation operators across SSIM thresholds from 0.50 to 1.00. Despite belonging to a mutation family that can be highly disruptive, these operators maintain consistently high prediction consistency across all SSIM levels. The lack of degradation at lower SSIM values and the stable performance at higher thresholds indicate that these permutation-based mutations do not adversely affect the classifier's predictions. This suggests that not all permutation-based

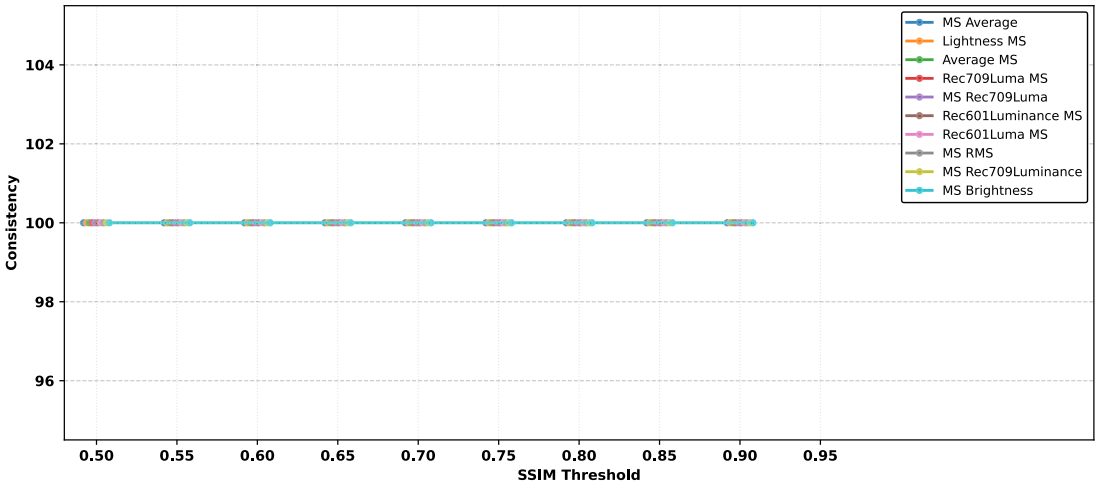


Fig. A.13. Top 10 least effective mutation operators in case of pair-wise permutation filter using SSIM.

transformations lead to robustness failures, and certain configurations remain well within the classifier's tolerance across the evaluated SSIM range.

Data availability

Artifacts include responses generated by a closed source tool used during its internal testing for a company. Other relevant components of the artifacts are publicly available at: https://osf.io/fau7k.

References

[1] Ideogram, Ideogram ai: Generated image by ideogram, 2025, https://ideogram.ai/g/0fl3AAypQo2GPB1eFtA-hQ/1.

[2] T. Kumar, R. Brennan, A. Mileo, M. Bendechache, Image data augmentation approaches: A comprehensive survey and future directions, 2024, http://dx.doi.org/10.1109/ACCESS.2024.3470122.

[3] V. Asnani, X. Yin, T. Hassner, S. Liu, X. Liu, Proactive image manipulation detection, 2022, http://dx.doi.org/10.1109/CVPR52688.2022.01495.

[4] K. Eykholt, I. Evtimov, E. Fernandes, B. Li, A. Rahmati, C. Xiao, A. Prakash, T. Kohno, D. Song, Robust physical-world attacks on deep learning visual classification, in: 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2018, pp. 1625–1634, http://dx.doi.org/10.1109/CVPR.2018.00175.

[5] S.G. Finlayson, I.S. Kohane, A.L. Beam, Adversarial attacks against medical deep learning systems, 2018, CoRR abs/1804.05296 arXiv:1804.05296, http://arxiv.org/abs/1804.05296.

[6] K. Hill, Wrongfully accused by an algorithm, 2020, https://www.nytimes.com/2020/06/24/technology/facial-recognition-arrest.html, the New York Times.

[7] J. Buolamwini, T. Gebru, Gender shades: Intersectional accuracy disparities in commercial gender classification, in: S.A. Friedler, C. Wilson (Eds.), Proceedings of the 1st Conference on Fairness, Accountability and Transparency, in: Proceedings of Machine Learning Research, PMLR, Vol. 81, 2018, pp. 77–91, https://proceedings.mlr.press/v81/buolamwini18a.html.

[8] T. Zahavy, A. Magnani, A. Krishnan, S. Mannor, Is a picture worth a thousand words? a deep multi-modal fusion architecture for product classification in e-commerce, 2016, arXiv:1611.09534, https://arxiv.org/abs/1611.09534.

[9] K. Pei, Y. Cao, J. Yang, S. Jana, Deepxplore: Automated whitebox testing of deep learning systems, in: Proceedings of the 26th Symposium on Operating Systems Principles, SOSP'17, ACM, 2017, pp. 1–18, http://dx.doi.org/10.1145/3132747.3132785.

[10] K. Zhang, Z. Cao, J. Wu, Circular shift: An effective data augmentation method for convolutional neural network on image classification, in: 2020 IEEE International Conference on Image Processing, ICIP, 2020, pp. 1676–1680, http://dx.doi.org/10.1109/ICIP40778.2020.9191303.

[11] Tool Developers, Closed source tool, 2023.

[12] Z. Wang, A. Bovik, H. Sheikh, E. Simoncelli, Image quality assessment: from error visibility to structural similarity, IEEE Trans. Image Process. 13 (4) 600–612, http://dx.doi.org/10.1109/TIP.2003.819861.

[13] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A.C. Berg, L. Fei-Fei, Large scale visual recognition challenge, Int. J. Comput. Vis. 115 (3) (2015) 211–252, http://dx.doi.org/10.1007/s11263-015-0816-y.

[14] D.R. Ivanova, I. Ilievski, M.P. Konstantinov, Towards more rigorous evaluations of language models, in: The Fourth Blogpost Track at ICLR 2025, 2025, https://openreview.net/forum?id=RaKOUPZmZj.

[15] Y. Tian, Z. Zhong, V. Ordonez, G. Kaiser, B. Ray, Testing dnn image classifiers for confusion &; bias errors, in: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering, ICSE'20, ACM, 2020, pp. 1122–1134, http://dx.doi.org/10.1145/3377811.3380400.

[16] X. Xie, J.W. Ho, C. Murphy, G. Kaiser, B. Xu, T.Y. Chen, Testing and validating machine learning classifiers by metamorphic testing, J. Syst. Softw. 84 (4) (2011) 544–558, http://dx.doi.org/10.1016/j.jss.2010.11.920, the Ninth International Conference on Quality Software. https://www.sciencedirect.com/science/article/pii/S0164121210003213.

[17] T. Jameel, M. Lin, L. Chao, Metamorphic relations based test oracles for image processing applications, Int. J. Softw. Innov. 4 (1) (2016) 16–30.

[18] A. Helin, Radamsa: A general-purpose fuzzer, https://gitlab.com/akihe/radamsa.

[19] K. Padmavathi, K. Thangadurai, Implementation of rgb and grayscale images in plant leaves disease detection – comparative study, Indian J. Sci. Technol. 9 (2016) http://dx.doi.org/10.17485/ijst/2016/v9i6/77739.

[20] M. Jaderberg, K. Simonyan, A. Zisserman, K. Kavukcuoglu, Spatial transformer networks, in: Proceedings of the 29th International Conference on Neural Information Processing Systems - Volume 2, NIPS'15, MIT Press, Cambridge, MA, USA, 2015, pp. 2017–2025.

[21] A. Azulay, Y. Weiss, Why do deep convolutional networks generalize so poorly to small image transformations? J. Mach. Learn. Res. 20 (184) (2019) 1–25, http://jmlr.org/papers/v20/19-519.html.

[22] S. Dodge, L. Karam, A study and comparison of human and deep learning recognition performance under visual distortions, in: 2017 26th International Conference on Computer Communication and Networks, ICCCN, 2017, pp. 1–7, http://dx.doi.org/10.1109/ICCCN.2017.8038465.

[23] D. Hendrycks, T. Dietterich, Benchmarking neural network robustness to common corruptions and perturbations, 2019, arXiv:1903.12261, https://arxiv.org/abs/1903.12261.

[24] A. Fawzi, P. Frossard S.-M. Moosavi-Dezfooli, Robustness of classifiers: from adversarial to random noise, 2016, arXiv:1608.08967, https://arxiv.org/abs/1608.08967.

[25] C. Zhang, S. Bengio, M. Hardt, B. Recht, O. Vinyals, Understanding deep learning (still) requires rethinking generalization, Commun. ACM 64 (3) (2021) 107–115, http://dx.doi.org/10.1145/3446776.

[26] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, F.A. Wichmann, Shortcut learning in deep neural networks, Nat. Mach. Intell. 2 (11) (2020) 665–673, http://dx.doi.org/10.1038/s42256-020-00257-z.

[27] D. Hendrycks, K. Zhao, S. Basart, J. Steinhardt, D. Song, Natural adversarial examples, 2021, arXiv:1907.07174, https://arxiv.org/abs/1907.07174.

[28] Z. Zhao, D. Dua, S. Singh, Generating natural adversarial examples, 2018, arXiv:1710.11342, https://arxiv.org/abs/1710.11342.

[29] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, A. Vladu, Towards deep learning models resistant to adversarial attacks, 2017, https://api.semanticscholar.org/CorpusID:3488815.

[30] B. S. H. Park Jinbae, Kumar Teerath, Search for optimal data augmentation policy for environmental sound classification with deep neural networks, J. Broadcast Eng. Korea Science (2020) http://dx.doi.org/10.5909/JBE.2020.25.6.854, [Online; accessed 2025-02-05].

[31] M. Turab, T. Kumar, M. Bendechache, T. Saber, Investigating multi-feature selection and ensembling for audio classification, 2022, arXiv:2206.07511, https://arxiv.org/abs/2206.07511.

[32] A. Chandio, Y. Shen, M. Bendechache, I. Inayat, T. Kumar, Audd: Audio urdu digits dataset for automatic audio urdu digit recognition, Appl. Sci. 11 (19) (2021) http://dx.doi.org/10.3390/app11198842, https://www.mdpi.com/2076-3417/11/19/8842.

[33] X. Wang, K. Wang, S. Lian, A survey on face data augmentation for the training of deep neural networks, Neural Comput. Appl. 32 (19) (2020) 15503–15531, http://dx.doi.org/10.1007/s00521-020-04748-3.

[34] L. Perez, J. Wang, The effectiveness of data augmentation in image classification using deep learning, 2017, arXiv:1712.04621, https://arxiv.org/abs/1712.04621.

[35] I.J. Goodfellow, M. Mirza J. Pouget-Abadie, B. Xu, S. Ozair D. Warde-Farley, A. Courville, Y. Bengio, Generative adversarial networks, 2014, http://dx.doi.org/10.1145/3422622.

[36] A. Radford, L. Metz, S. Chintala, Unsupervised representation learning with deep convolutional generative adversarial networks, 2016, arXiv:1511.06434, https://arxiv.org/abs/1511.06434.

[37] T.O. Team, Gpt-4 technical report, 2024, arXiv:2303.08774, https://arxiv.org/abs/2303.08774.

[38] J. Betker, G. Goh, L. Jing, Tim Brooks, J. Wang, L. Li, Long Ouyang, Juntang Zhuang, Joyce Lee, Yufei Guo, Wesam Manassra, Prafulla Dhariwal, Casey Chu, Yunxin Jiao, A. Ramesh, Improving image generation with better captions, https://api.semanticscholar.org/CorpusID:264403242.

[39] Y. Yuan, Q. Pang, S. Wang, Provably valid and diverse mutations of real-world media data for dnn testing, IEEE Trans. Softw. Eng. 50 (5) (2024) 1040–1064, http://dx.doi.org/10.1109/TSE.2024.3370807.

[40] J. Zhu, Y. Shen, D. Zhao, B. Zhou, In-domain gan inversion for real image editing, 2020, http://dx.doi.org/10.1007/978-3-030-58520-4_35.

[41] Y. Shen, J. Gu, X. Tang, B. Zhou, Interpreting the latent space of gans for semantic face editing, 2020, https://api.semanticscholar.org/CorpusID:198897678.

[42] A. Hertzmann E. Härkönen, J. Lehtinen, S. Paris, Ganspace: Discovering interpretable gan controls, 2020, arXiv:2004.02546, https://arxiv.org/abs/2004.02546.

[43] H. Ling, K. Kreis, D. Li, S.W. Kim, A. Torralba, S. Fidler, Editgan: High-precision semantic image editing, 2021, arXiv:2111.03186, https://arxiv.org/abs/2111.03186.

[44] H. Kim, Y. Choi, J. Kim, S. Yoo, Y. Uh, Exploiting spatial dimensions of latent in gan for real-time image editing, 2021, http://dx.doi.org/10.1109/CVPR46437.2021.00091, https://doi.ieeecomputersociety.org/10.1109/CVPR46437.2021.00091.

[45] C. Du, Y. Li, Z. Qiu, C. Xu, Stable diffusion is unstable, 2023, arXiv:2306.02583, https://arxiv.org/abs/2306.02583.

[46] H. Chefer, Y. Alaluf, Y. Vinker, L. Wolf, D. Cohen-Or, Attend-and-excite: Attention-based semantic guidance for text-to-image diffusion models, 2023, http://dx.doi.org/10.1145/3592116.

[47] W. Feng, X. He, T.-J. Fu, V. Jampani, A. Akula, P. Narayana, S. Basu, X.E. Wang, W.Y. Wang, Training-free structured diffusion guidance for compositional text-to-image synthesis, 2023, arXiv:2212.05032, https://arxiv.org/abs/2212.05032.

[48] K. Alomar, H.I. Aysel, X. Cai, Data augmentation in classification and segmentation: A survey and new strategies, J. Imaging 9 (2) (2023) http://dx.doi.org/10.3390/jimaging9020046, https://www.mdpi.com/2313-433X/9/2/46.

[49] K. Maharana, S. Mondal, B. Nemade, A review: Data pre-processing and data augmentation techniques, Glob. Trans. Proc. 3 (1) (2022) 91–99, http://dx.doi.org/10.1016/j.gltp.2022.04.020, international Conference on Intelligent Engineering Approach(ICIEA-2022). https://www.sciencedirect.com/science/article/pii/S2666285X22000565.

[50] A. Krizhevsky, Learning multiple layers of features from tiny images, 2009, https://api.semanticscholar.org/CorpusID:18268744.

[51] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A.C. Berg, L. Fei-Fei, Imagenet large scale visual recognition challenge, 2015, arXiv:1409.0575, https://arxiv.org/abs/1409.0575.

[52] Y. Lecun, L. Bottou, Y. Bengio, P. Haffner, Gradient-based learning applied to document recognition, Proc. IEEE 86 (11) (1998) 2278–2324, http://dx.doi.org/10.1109/5.726791.